Exact Consistency Under Partial Views: Graph Colorability, Capacity, and Equality in Multi-Location Encodings

Tristan Simas

***Abstract*—We construct a structural theory of failure and a graph-capacity framework for analyzing structural integrity under distributed source coding. Structural integrity means the code can correct erasures: the mapping from source to observable syndrome is injective. Admissible partial views induce a confusability graph on latent tuples; in the exact coordinate-view model, this graph class is exactly characterized by upward-closed families of coordinate-agreement sets, and exact recovery with a T-ary tag is equivalent to T-colorability. Repeated composition yields strong powers, so the normalized block-rate sequence converges to asymptotic Shannon capacity bounded above by Lovász-ϑ. The upper theory is sharp whenever confusability is transitive; meet-witnessing and fiber coherence provide checkable sufficient conditions for that collapse. Under an affine restriction, the coordinate structure yields a representable matroid whose rank bounds confusability. The theory yields verifiable structural integrity criteria with applications to programming-language runtimes, databases, and dependency managers.**

I. Introduction

The substantive question in distributed source coding with side information is what exact ambiguity structure survives once a system exposes only partial observations of its source, and what formal structural integrity guarantees are possible in that regime. Structural integrity here means the code can correct erasures: the mapping from source to observable syndrome is injective, so the latent state is uniquely recoverable from the observable tag. This paper answers that question with a zero-error graph-capacity theory for deterministic partial views on latent tuples.

The theory is predictive: given only the view-family specification, the full failure topology, exact recovery law, asymptotic capacity, and standard upper bounds are deductive consequences of the architecture, with the realizable graph class fully characterized.

When one latent fact or tuple of facts is represented at several modifiable locations, the available observation need not identify a unique latent state. Under partial views, the surviving ambiguity has structure: the admissible view family induces a confusability graph on latent tuples. Its edges record the exact state pairs that the architecture cannot separate, proper colorings quantify the side-information budget needed for exact recovery, and repeated composition pushes the same structure to strong powers and asymptotic Shannon capacity. The paper therefore studies not just whether a multi-location system can fail, but the topology of those failures, the integrity guarantees excluded or permitted by that topology, and the recovery laws forced by it. The resulting theory lies at the intersection of zero-error information theory, side-information source coding, finite converses, and structural integrity analysis [1]–[6].

T. Simas is with McGill University, Montreal, QC, Canada (e-mail: tristan.simas@mail.mcgill.ca).

Manuscript received March 16, 2026.

Novelty. Unlike Witsenhausen's setting where the observation law is given, here a deterministic multi-location partial-view architecture generates that law and thereby constrains the realizable graph class.*(L: MFT125-131)*

The graph-generation mechanism is now more sharply characterized. On the full labeled tuple space, the exact coordinate-view model realizes exactly the confusability relations determined by upward-closed families of coordinate-agreement sets: two distinct tuples are adjacent if and only if their agreement set lies in such a family, and conversely every upward-closed family is realizable by some view architecture. *(L: MFT125-128)* When the alphabet has at least two symbols, every proper agreement set occurs for some distinct tuple pair, so this is a complete characterization up to the vacuous full-agreement case of identical tuples.*(L: MFT129-131)* Coordinatewise alphabet permutations still act by graph automorphisms, and any latent tuple can be carried to any other by such an automorphism.*(L: MFT120-124)* Hence the induced graphs form a specific monotone agreement-set class rather than an arbitrary finite graph family. The model already generates non-clique graphs, includes the cluster-collapse subclass on which equality is explicit, and is closed under the block-composition operation that becomes strong product on confusability graphs. The binary square already yields a 4-cycle rather than a clique, and its iterates give a concrete infinite non-clique strong-product family inside the class.

A. Distributed Source Coding Formulation

The model yields an exact-consistency analogue of zero-error resolvability for modifiable encodings. Rate is measured by the number of independently writable locations; we refer to this quantity as the *independent rate* and use *degrees of freedom* (DOF) only as shorthand.

Failure geometry from partial views. The main jump of the paper is from clique-shaped ambiguity to architecture-specific failure structure. In the multi-fact partial-observation

extension, the confusability graph is the structural object controlling exact consistency under partial views, exact recovery is equivalent to graph colorability, and exact finite weighted success is determined by the maximum T-colorable induced subgraph. The binary two-fact square already shows that the induced graph need not be a clique: it is a 4-cycle, so failures above the unit-rate boundary are structured rather than uniform. *(L: MFT3, MFT5–10, MFT24)*

Asymptotic rate and upper theory. This graph characterization then lifts to strong powers and asymptotic zero-error rates. The full n-block confusability graph is the n-fold strong power of the one-shot graph; the normalized block-rate sequence converges to a real asymptotic Shannon-capacity value equal to the supremum of the finite block rates; and that value is bounded above by both the logarithm of the complement chromatic number and a fixed Lovász-ϑ upper convention. The one-shot upper object matches standard orthonormal-representation, primal-PSD, and dual-theta forms. *(L: MFT35, MFT48, MFT55, MFT64, MFT69, MFT84)*

Equality characterization. The paper also proves an equality characterization. For the original clique-fiber subclass coming from a surjective label map, asymptotic Shannon capacity and the fixed Lovász-ϑ upper both collapse to $\log|\mathcal{B}|$, where $\mathcal{B}$ is the fiber-label alphabet. This equality for cluster-graph structure is classical; the new point here is that transitivity of base confusability gives a model-side route from the view-family architecture to the cluster-graph equality case, while meet-witnessing and fiber coherence are stronger sufficient conditions. Under these conditions, the asymptotic Shannon capacity and the fixed Lovász-ϑ upper collapse to the logarithm of the number of connected components or realized transcript fibers, depending on the structure available. *(L: MFT85, MFT91, MFT96, MFT102, MFT109)*

Finite converse foundation. Let X be a finite latent source, let Y be the deterministic observation transcript available to a resolver, and let T be a finite auxiliary tag. Exact zero-error recovery from (Y,T) is possible exactly when the pair map $x \mapsto (Y(x),T(x))$ is injective on the surviving ambiguity class. The same obstruction appears as confusability, counting, conditional-entropy, decoder-output, and finite-gap formulations, and it admits a budgeted finite-error extension. In particular, exact k-way resolution requires at least $\log_2 k$ bits of side information. *(L: OBS1–2, OBS5, CIA3, PRB47, PRB55, PRB68, PRB81, PRB83, PRB85, PRB87, PRB90)*

Boundary corollary. For deterministic multi-location encodings, the zero-incoherence threshold equals 1: the single-source regime is the unique no-failure corner, and it is exactly the structural-integrity regime. In coding-theoretic terms, this is the erasure-correcting regime where the syndrome uniquely determines the message. The richer mathematics begins once the finite obstruction survives and acquires nontrivial confusability structure. *(L: COH1–2, RAT1, CAP2–3)*

Secondary fact-side question. The same framework also supports a second structural question: which fact coordinates are determined by others across the realized state family? Under an affine restriction, that coordinate-dependence question becomes the standard span-membership condition on coordinate functionals, so the induced fact-closure is a representable matroid on fact indices. The value of this specialization here is operational rather than linear-algebraic novelty: the same realized-state model then carries two complementary invariants, a graph on latent states and, in the affine regime, a matroid on coordinates whose rank gives upper bounds on confusability and capacity. Those bounds are algorithmically tractable once the affine family is presented explicitly by a basis or generator matrix for its direction space; under that representation, the relevant rank quantity is exactly the rank of the restricted coordinate projection and satisfies formal size bounds such as $t(S) \le |S|$, so the computation reduces to Gaussian elimination. *(L: AFM1–11)*

B. Realizability and Verification

The same deterministic model also raises a realizability question at its opposite boundary: when can a concrete host system realize the rate-1 corner and thereby certify structural integrity? We show that two structural properties are required:

1) **Causal update propagation:** derived locations must update automatically when the source changes.
2) **Provenance observability:** the system must expose enough structural information to verify which locations are authoritative and which are derived.

C. Paper Organization

The main theorem arc is: partial views induce a confusability graph; exact recovery becomes graph colorability; block composition turns that graph into strong powers; the resulting normalized rates converge to the classical Shannon-capacity value of the induced graph; a fixed Lovász-ϑ upper theory bounds that value; and transitivity of confusability gives the model-side export to the classical cluster-graph equality case. The deterministic converse is the finite foundation of that arc. The affine and realizability sections are later specializations and boundary consequences of the same model rather than separate primary claims.

Section III gives the minimum model and threshold setup. Section IV develops the finite converse foundation. Sections V–VIII contain the main graph-capacity arc: graph characterization, block and asymptotic capacity, upper theory, and equality characterization. Section IX then records the affine fact-side specialization as a second lens on the same confusability structure. Section X, Section XI, and Section XII return to the unit-rate boundary of the same model and develop its structural, operational, and rate-complexity consequences. Appendix A records representative host-level readings, and Supplement A contains case-study details and traces. A supplementary Lean 4 artifact machine-checks the converse family, graph extension, theta upper theory, affine fact-matroid specialization, threshold chain, operational realizability criterion, and rate-complexity arguments [7].

D. Scope

The scope is finite facts represented at multiple locations under admissible edits. The focus is on *structural facts*, meaning facts whose encoding locations are fixed when an

object, declaration, or artifact is created. This isolates the zero-error regime and captures a wide class of realizations without making the theory domain-specific.

E. Contributions

The main contributions are grouped into the core graph-capacity arc and the application consequences built on the same model.

a) Core graph-capacity contributions.:

- **A multi-fact confusability-graph extension** in which partial observations on latent tuples produce non-clique confusability graphs, exact recovery becomes equivalent to graph colorability, success-set exactness becomes equivalent to induced-subgraph colorability, and the exact finite weighted-success value is characterized by the maximum T-colorable induced subgraph.
- **A strong-power block law and asymptotic Shannon-capacity export** in which the n-block confusability graph is the n-fold strong power of the one-shot graph, so the classical Shannon-capacity framework applies directly to the induced graph family and the normalized block-rate sequence converges to the corresponding asymptotic capacity value.
- **A fixed Lovász-ϑ upper theory and standard equivalent forms** in which the same asymptotic capacity is bounded above by complement-chromatic and fixed Lovász-ϑ upper objects, with standard orthonormal, primal-PSD, and dual-theta forms.
- **A model-side equality characterization** in which the classical cluster-graph equality case is exported back to the original view-family model under transitivity of confusability, with meet-witnessing as a checkable sufficient condition and fiber coherence as a stronger one.
- **A deterministic finite converse toolkit** for exact consistency, presented through pair injectivity together with confusability, counting, conditional-entropy, decoder-output, and entropy-gap formulations of the same finite obstruction, deterministic data processing, and a budgeted finite-error extension.

b) Consequence contributions.:

- **An affine fact-matroid specialization from the same latent-state framework** in which affine realized-state families induce a representable matroid on fact indices: semantic determination becomes span membership, and the resulting matroid rank yields tractable upper bounds for the main confusability/capacity problem.
- **Threshold and realizability corollaries** showing that unit rate is the unique zero-incoherence regime, that unit rate is exactly the structural-integrity regime, that unit rate has $O(1)$ manual update cost while higher rates incur an $\Omega(n)$ lower bound, and that causal propagation together with provenance observability gives an operational criterion for realizable verifiable structural integrity in concrete hosts.
- **A supplementary machine-checked artifact** verifying the theorem chain in Lean 4.

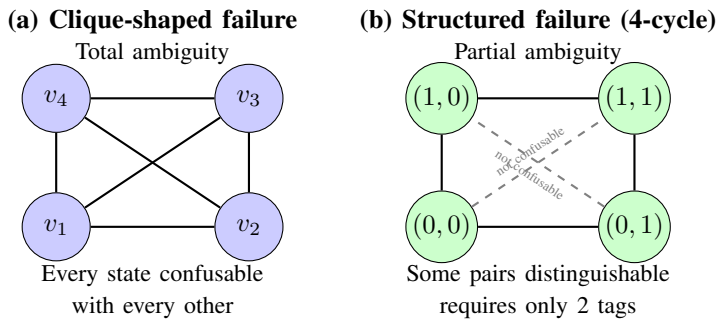

Fig. 1. Two failure topologies. In (a), the observation provides no discrimination, so the confusability graph is a clique: every state is confusable with every other, and exact recovery requires a tag for each state. In (b), partial observations provide some discrimination: opposite corners are distinguishable even though adjacent states are not. The 4-cycle is 2-colorable, so exact recovery requires only 2 tags. This structural difference is invisible to the unit-rate threshold alone.

II. Figures

III. Model and Basic Quantities

This section introduces only the minimum structure needed to state the deterministic converse foundation and its boundary threshold corollary: latent states, deterministic observations, auxiliary tags, ambiguity classes, and independent rate.

A. Model assumptions and notation

We use the following standing assumptions.

1) **Finite latent state:** each fact takes values in a finite alphabet.
2) **Deterministic observations:** a system state exposes a deterministic observation transcript.
3) **Independent rate:** the independent rate counts the number of independently writable source locations for the same latent fact.
4) **Finite side information:** any auxiliary tag is drawn from a finite alphabet.

These are the only ingredients needed for the deterministic zero-error theory developed below.

B. Latent states, ambiguity, and independent rate

An *encoding system* for a fact F is a finite collection of locations $\{L_1, \ldots, L_n\}$ whose current values jointly determine the observation transcript available to a resolver. For a system state x, the *ambiguity class* is the set of latent values still compatible with that transcript; when the ambiguity class has size greater than one, the transcript alone does not identify a unique authoritative value. In this single-fact setting, latent-state ambiguity and value ambiguity coincide because the latent state is just the fact value. In the later multi-fact extension, ambiguity is instead over full latent tuples compatible with the partial observations. A *fact* is an atomic semantic attribute of the represented object that can vary independently of other attributes, and a fact is *structural* if the locations encoding it are fixed when the underlying object, declaration, or artifact is created.

The admissible edit set $E(C)$ is part of the primitive specification of an encoding system C. It records which edit events or edit sequences are considered reachable by the model. The paper does not derive $E(C)$ from a particular concurrency policy, protocol, or consensus mechanism; those are downstream instantiations. Independence and independent rate are therefore always defined *relative to the chosen admissible edit model*.

Definition III.1 (Independent Location)**.** Let $E(C)$ denote the admissible edit set of encoding system C. Two locations encoding the same fact are *independent relative to* $E(C)$ if some admissible edit sequence in $E(C)$ can change one without forcing the other to match it, equivalently if some admissible edit sequence reaches a state in which the two locations disagree.

Definition III.2 (Derived Location)**.** A location is *derived* from a source location if updating the source determines the derived value without an additional manual edit.

Definition III.3 (Degrees of Freedom)**.** The *degrees of freedom* (DOF), or *independent rate*, of an encoding system for fact F is the number of pairwise independent source locations encoding F under the specified admissible edit set $E(C)$. In the single-fact model, the remaining encoding locations are treated as derived views of those independent sources.

Definition III.4 (Structural Integrity)**.** An encoding system has *structural integrity* for fact F if every reachable state is coherent, so no reachable system state presents mutually incompatible encodings of F.

Definition III.5 (Integrity Violation)**.** A reachable *integrity violation* is a reachable incoherent state for fact F.

Proposition III.6 (Unit Independent Rate Guarantees Coherence)**.** *If the independent rate is* 1*, then all reachable states are coherent.*

(L: COH1)

Proof. At independent rate 1, exactly one location is independent and all remaining locations are derived from it. Derived locations cannot diverge from their source, so disagreement is unreachable. ■

Proposition III.7 (Independent Rate Above One Permits Incoherence)**.** *If the independent rate exceeds* 1*, then incoherent states are reachable.*

(L: COH2)

Proof. With at least two independent locations, admissible edits can assign different values to those locations, producing a reachable disagreement state. ■

C. Zero-Incoherence Threshold

Definition III.8 (Zero-Incoherence Threshold)**.** The *zero-incoherence threshold* is the largest independent rate for which incoherent states are unreachable.

Corollary III.9 (Threshold Corollary)**.** *For deterministic multi-location encodings, the zero-incoherence threshold equals* 1*.*

(L: RAT1, CAP2-3)

Proof. Achievability. Suppose $\mathrm{DOF}(C, F) = 1$, and let L_s be the unique independent location. By Definition III.3, every other encoding location is non-independent and is therefore treated in this model as a derived view of L_s. By Definition III.2, updating L_s determines the value of each such derived location without an additional manual edit. Hence all encoding locations agree in every reachable state, so zero incoherence is achieved.

Converse. Suppose $\mathrm{DOF}(C, F) = k > 1$. By Definition III.1, there exist at least two independent locations L_1, L_2 and an admissible edit sequence that can make them disagree. Equivalently, choose values $v_1 \neq v_2$ and apply edits setting $L_1 \leftarrow v_1$ and then $L_2 \leftarrow v_2$. By independence, the second edit does not force L_1 to change. The resulting reachable state satisfies $\mathrm{value}(L_1) \neq \mathrm{value}(L_2)$ and is therefore incoherent.

Thus zero-error recovery is possible exactly at independent rate 1, so the zero-incoherence threshold equals 1. ■

Corollary III.10 (Exact Structural-Integrity Threshold)**.** *A deterministic multi-location encoding has structural integrity if and only if its independent rate equals* 1*.*

(L: COH1-2)

Proof. If the independent rate is 1, Proposition III.6 shows that all reachable states are coherent, hence Definition III.4 holds. If the independent rate exceeds 1, Proposition III.7 gives a reachable incoherent state, so structural integrity fails. ■

This threshold is an early architectural consequence of the deterministic model. It identifies the unique trivial no-failure corner and, equivalently, the exact structural-integrity regime. Above that boundary, integrity violations are reachable by construction. The deeper theorems of the paper begin once one asks how much finite side information is needed whenever the transcript leaves a nontrivial ambiguity class.

Corollary III.11 (Side-Information Lower Bound)**.** *If a surviving ambiguity class has size k, then exact zero-error resolution requires at least $\log_2 k$ bits of side information.*

(L: CIA1)

Proof. Exact resolution of k surviving alternatives requires at least k distinguishable tags, hence at least $\log_2 k$ bits. ■

The remainder of the paper sharpens this counting statement into a unified finite converse family, asks how that one-shot obstruction acquires non-clique topology under partial views, and then develops the corresponding asymptotic capacity and upper-bound theory.

IV. Unified Finite Converse

This section supplies the one-shot obstruction that the later graph-capacity theory lifts from clique-shaped ambiguity to non-clique failure geometry. Its role is foundational and unifying rather than graph-theoretically novel: in the single-fact

setting, every surviving ambiguity class is effectively clique-shaped under the observation transcript, so the same deterministic obstruction can be stated as injectivity, clique counting, entropy, decoder-output, and finite-error bounds. The cleanest zero-error formulation starts from the joint observation-tag map itself. The first theorem gives the basic criterion; the remaining statements package the same obstruction in the forms reused later in the graph and realizability arcs.

Theorem IV.1 (Pair-injectivity characterization)**.** *Let X range over a finite latent alphabet, let Y be the deterministic observation transcript, and let T be a finite auxiliary tag. Exact zero-error recovery from (Y, T) is possible if and only if the pair map $x \mapsto (Y(x), T(x))$ is injective on the ambiguity class under consideration.*

(L: OBS1–2)

Proof. Let $\phi(x) := (Y(x), T(x))$.

Necessity. If ϕ is not injective, then there exist distinct latent states $x_1 \neq x_2$ with $\phi(x_1) = \phi(x_2)$. Any deterministic decoder D therefore receives the same input on both states and must satisfy $D(\phi(x_1)) = D(\phi(x_2))$. Since $x_1 \neq x_2$, the common output cannot equal both latent states, so D errs on at least one of them.

Sufficiency. If ϕ is injective, define a decoder on the image of ϕ by $D(y, t) = \phi^{-1}(y, t)$, with arbitrary extension off the image. Injectivity makes ϕ^{-1} single-valued on the image, and for every latent state x we then have $D(Y(x), T(x)) = x$. Thus exact zero-error recovery is possible. ■

Proposition IV.2 (Confusability formulation)**.** *Fix a deterministic observation transcript and an L-bit side-information tag. If K latent states induce the same observation transcript, then exact zero-error decoding on that ambiguity class requires K distinct tag outcomes. Equivalently, a K-way ambiguity class forms a confusability clique whose size cannot exceed the available tag alphabet.*

(L: OBS5)

Proof. Apply Theorem IV.1 on an ambiguity class on which the observation is constant. Then injectivity of (Y, T) reduces to injectivity of the tag coordinate alone on that class. Since an L-bit tag provides at most 2^L outcomes, the clique size cannot exceed 2^L. ■

a) Global observation-tag budget.: If the observation alphabet has size O and the auxiliary tag alphabet has size T, then exact zero-error recovery on a latent alphabet of size K requires

$$K \leq OT.$$

(L: OBS4)

Proof. Let $\phi(x) = (Y(x), T(x))$. By Theorem IV.1, exact zero-error recovery requires ϕ to be injective on the latent alphabet under consideration. The codomain of ϕ is the finite product alphabet $\mathcal{Y} \times \mathcal{T}$, which has cardinality $|\mathcal{Y} \times \mathcal{T}| = OT$. An injective map from a set of size K into a set of size OT can exist only if $K \leq OT$. This is exactly the claimed global budget bound. ■

b) Fiber-level injectivity and cardinality.: Fix an observation value y. On the observation fiber

$$\mathcal{F}_y = \{x : Y(x) = y\},$$

exact zero-error recovery forces the tag map to be injective. Consequently,

$$|\mathcal{F}_y| \leq |\mathcal{T}|.$$

(L: OBS3)

Proof. Fix y and restrict attention to the fiber $\mathcal{F}_y = \{x : Y(x) = y\}$. On this set the observation coordinate is constant, so for $x, x' \in \mathcal{F}_y$ one has

$$(Y(x), T(x)) = (Y(x'), T(x')) \iff T(x) = T(x').$$

Hence Theorem IV.1 implies that exact recovery on $\mathcal{F}_y$ is possible only if the tag map $x \mapsto T(x)$ is injective on that fiber. Since the image of an injective map cannot be larger than its codomain, one obtains $|\mathcal{F}_y| \leq |\mathcal{T}|$. ■

Proposition IV.3 (Finite counting converse)**.** *Let F range over K possible latent values. Suppose exact recovery is attempted from a fixed observation transcript together with an L-bit side-information tag. Then*

$$K \leq 2^L.$$

Equivalently, exact zero-error recovery of K ambiguous states requires at least $\log_2 K$ bits of side information.

(L: CIA3)

Proof. This is Theorem IV.2 specialized to a single K-way ambiguity class. ■

The counting bound is the coarsest form of the obstruction. The next step is to weight the same argument by source mass. Exact recovery still enforces injectivity on successful confusability classes, but now the question is how much entropy can remain once the source is partitioned into success and failure branches.

c) Conditional-entropy formulation.: Let X be uniform on a K-way ambiguity class and let Y be the deterministic observation transcript. If Y is constant on that class, then $H(X \mid Y) = \log_2 K$. In particular, any tag-observation pair (Y, T) that resolves the class exactly must satisfy

$$H(X \mid Y, T) = 0 \qquad \text{and} \qquad H(T \mid Y) \geq \log_2 K.$$

(L: PRB47, PRB55)

Proof. If Y is constant on the K-point ambiguity class, then conditioning on Y does not refine the distribution of X. Because X is uniform on that class, this gives $H(X \mid Y) = \log_2 K$. Exact recovery from (Y, T) means that X is a deterministic function of (Y, T), so $H(X \mid Y, T) = 0$. Using the chain rule,

$$H(X \mid Y) = I(X; T \mid Y) + H(X \mid Y, T) = I(X; T \mid Y).$$

Therefore $I(X; T \mid Y) = \log_2 K$. Since conditional mutual information is bounded by conditional entropy, $I(X; T \mid Y) \leq H(T \mid Y)$, and the lower bound $H(T \mid Y) \geq \log_2 K$ follows. ■

d) Mass-weighted clique entropy bound.: Let S be a success set for a zero-error decoder, and suppose S forms a confusability clique under the observation transcript. If p_S denotes the total probability mass of S, then the entropy carried by the successful states obeys

$$\sum_{x\in S} p(x)\log_2\frac{1}{p(x)} \;\le\; h_2(p_S) + p_S\log_2|\mathcal{T}|,$$

where h_2 is binary entropy and $|\mathcal{T}|$ is the tag alphabet size. *(L: PRB55)*

Proof. Let B be the indicator of the event $\{X \in S\}$. Then $\Pr[B=1] = p_S$, so $H(B) = h_2(p_S)$. Decompose the source entropy carried by the successful states by conditioning on B:

$$H(X) = H(B) + H(X \mid B).$$

Restricting to the success branch $B = 1$, exact recovery on the clique S forces injectivity of the tag map on S, hence at most $|\mathcal{T}|$ successful states can be resolved. Therefore

$$H(X \mid B=1) \le \log_2|\mathcal{T}|.$$

Multiplying by the success probability gives a contribution of at most $p_S\log_2|\mathcal{T}|$ from that branch. Combining this with the Bernoulli split term $h_2(p_S)$ yields the displayed inequality for the entropy mass carried by the successful states. ■

In particular, on any exact success set inside a confusability clique, the restricted observation-tag map is injective by Theorem IV.1, so the same entropy inequality applies directly to the successful branch rather than only to the whole clique. *(L: OBS1, PRB55)*

e) Decoder-output and gap formulations.: In the same deterministic finite model, the obstruction can also be expressed as output-entropy and finite-budget gap constraints: any deterministic decoder output $\hat{X}$ satisfies

$$H(\hat{X}) \le H(Y,T) \le H(X),$$

and the observation-tag entropy and decoded-output entropy are bounded by their corresponding finite alphabet ceilings, so the deficits

$$\log_2|\mathcal{Y}\times\mathcal{T}| - H(Y,T), \qquad \log_2|\widehat{\mathcal{X}}| - H(\hat{X})$$

are nonnegative and vanish only in the uniform saturation cases formalized in the supplement. *(L: PRB73–74, PRB82–83, PRB90, PRB93–95)*

Proof. Because the decoder is deterministic, $\hat{X} = D(Y,T)$ is a deterministic function of the pair (Y,T). The deterministic data-processing statement below therefore gives

$$H(\hat{X}) \le H(Y,T).$$

The pair (Y,T) is itself a deterministic function of the latent source X, so another application of deterministic data processing yields $H(Y,T) \le H(X)$. Since (Y,T) takes values in the finite alphabet $\mathcal{Y}\times\mathcal{T}$, one also has

$$H(Y,T) \le \log_2|\mathcal{Y}\times\mathcal{T}|.$$

Likewise $H(\hat{X}) \le \log_2|\widehat{\mathcal{X}}|$. Subtracting these entropies from their respective alphabet ceilings gives the nonnegative gap quantities claimed in the statement. ■

f) Deterministic data processing.: Let κ be any deterministic coarsening of the observation-tag pair (Y,T). Then

$$H(\kappa(Y,T)) \le H(Y,T).$$

In particular, since any deterministic decoder output $\hat{X}$ is a coarsening of (Y,T),

$$H(\hat{X}) \le H(Y,T) \le \log_2|\mathcal{Y}\times\mathcal{T}|.$$

(L: PRB68, PRB81, PRB83)

Proof. Write $Z = (Y,T)$. If κ is deterministic, then $\kappa(Z)$ is a function of Z, so the conditional entropy $H(\kappa(Z) \mid Z)$ is zero. By the chain rule,

$$H(Z) = H(\kappa(Z), Z) = H(\kappa(Z)) + H(Z \mid \kappa(Z)),$$

and since conditional entropy is nonnegative, $H(\kappa(Z)) \le H(Z)$. Taking κ to be the decoder map gives $H(\hat{X}) \le H(Y,T)$. Finally, because Z ranges over the finite alphabet $\mathcal{Y}\times\mathcal{T}$, one has the standard finite-alphabet ceiling $H(Y,T) \le \log_2|\mathcal{Y}\times\mathcal{T}|$. ■

This theorem is the structural reason the output and gap formulations are not independent embellishments of the counting converse. Once the observation is coarsened, every derived representation inherits the same obstruction through deterministic data processing rather than escaping it. *(L: PRB68, PRB81, PRB83, PRB90)*

Proposition IV.4 (Equivalence viewpoint)**.** *The confusability, counting, conditional-entropy, decoder-output, and finite-gap statements above are different normal forms of the same deterministic finite zero-error obstruction: a surviving K-way ambiguity class requires a budget of at least $\log_2 K$ bits to be resolved exactly.*

(L: OBS1, OBS5, CIA3, PRB47, PRB55, PRB68, PRB81, PRB83, PRB90)

Proof. Pair injectivity is the structural core. The confusability and counting theorems express the injectivity requirement combinatorially; the conditional-entropy and weighted-entropy theorems express it in source-mass coordinates; the decoder-output and gap theorems express it through deterministic coarsening and finite alphabet ceilings; and the finite-error theorem is the relaxed version obtained by allowing a nonzero failure branch. Each theorem therefore measures the same failure of exact isolation in a different coordinate system. ■

A. Finite-error extension

The main body of the paper studies exact zero error. The same deterministic finite model also supports a budgeted finite-error extension, which should be read as a deterministic finite analogue of the classical Fano line of argument rather than as a separate asymptotic coding theorem [6], [8], [9].

Proposition IV.5 (Budgeted finite-error converse)**.** *Let P_e denote the decoder error probability on a finite source of size K, observed through a deterministic transcript alphabet of size O together with a tag alphabet of size T. Then the decoded-output entropy obeys*

$$H(\hat{X}) \;\le\; h_2(P_e) + (1-P_e)\log_2(OT) + P_e\log_2(K-1).$$

(L: PRB85, PRB87)

Proof. Partition the source into success and failure events. On the success branch, the decoded output is constrained by the observation-tag budget and contributes at most $\log_2(OT)$. On the failure branch, the decoder can still output at most one of the remaining $K-1$ alternatives. The Bernoulli split between the two branches contributes the binary-entropy term. Equivalently,

$$H(\hat{X}) = H(\hat{X} \mid \text{success/failure}) + H(\text{success/failure})$$

is bounded by combining the support bound on each branch with the binary entropy of the branch variable itself. ■

The zero-error theorems are the $P_e = 0$ boundary of this inequality. In that regime the success branch occupies all probability mass, the Bernoulli term vanishes, the failure contribution disappears, and the budget collapses back to the unified finite converse above.

The next section asks what survives when admissible views separate some latent alternatives but not others. In that regime the same one-shot obstruction is no longer represented by a single clique; it becomes an induced confusability graph that can carry genuinely non-clique failure structure.

V. Graph Characterization of Partial Views

Before defining the general confusability graph, we develop a concrete example: partial observations can induce nontrivial failure topology.

A. A Running Example: The Binary Square

Consider a system with two binary facts $(x_1, x_2) \in \{0,1\}^2$, giving four latent states:

$$(0,0), \quad (0,1), \quad (1,0), \quad (1,1).$$

Suppose the architecture exposes two partial views:

- View 1 reveals only coordinate x_1
- View 2 reveals only coordinate x_2

a) Confusability structure.: Two states are confusable when at least one view fails to separate them:

- Through View 1: states $(0,0)$ and $(0,1)$ are confusable (both have $x_1 = 0$), and states $(1,0)$ and $(1,1)$ are confusable (both have $x_1 = 1$).
- Through View 2: states $(0,0)$ and $(1,0)$ are confusable (both have $x_2 = 0$), and states $(0,1)$ and $(1,1)$ are confusable (both have $x_2 = 1$).

The confusability graph is therefore a 4-cycle:

$$(0,0) \sim (0,1) \sim (1,1) \sim (1,0) \sim (0,0),$$

where opposite corners $(0,0)$ and $(1,1)$, as well as $(0,1)$ and $(1,0)$, are *not* adjacent. This is not a clique: the architecture can distinguish some pairs even though it cannot distinguish all pairs.

b) Exact recovery.: Because the graph is 2-colorable (e.g., color by parity $c(x_1, x_2) = x_1 \oplus x_2$), exact recovery is possible with a 2-ary auxiliary tag. However, the graph is not 1-colorable (it contains edges), so a 1-tag decoder must fail on at least one confusable pair. Under the uniform source, the best 1-tag exact success probability is $1/2$ (choose either diagonal as the success set).

c) What this example shows.:

1) Partial views induce *structured* confusability, not uniform ambiguity.
2) The confusability graph captures exactly which state pairs the architecture cannot separate.
3) Graph-theoretic properties (chromatic number, independence number) determine exact recovery costs.

Coding-theoretic interpretation. This 4-cycle structure has a direct interpretation in error-correcting codes. Coloring by parity $c(x_1, x_2) = x_1 \oplus x_2$ is exactly a syndrome: the tag reveals whether the two bits match. With one syndrome bit ($T = 2$), we can decode perfectly because each color class contains non-adjacent states. With no tag ($T = 1$), we cannot distinguish adjacent vertices, exactly as in the classic binary symmetric channel without feedback. The confusability graph is precisely the confusion graph of a code with parity-check matrix $H = [1\ 1]$: adjacent vertices are those the code cannot separate.

Figure 1 contrasts this structured failure with the clique-shaped failure that arises when no observation provides any discrimination. The theorems below generalize this phenomenon to arbitrary multi-fact partial-view systems.

B. General Graph Characterization

This section turns the one-shot obstruction into the paper's main structural object: the confusability graph induced by admissible partial views. The single-fact converse family treats ambiguity classes that are effectively clique-shaped under the observation transcript. One extension is to let the latent state be a tuple of facts and let each location reveal only a subset of coordinates. Failure then becomes structured: some latent pairs remain indistinguishable and others do not, so the resulting confusability graph need not be complete.

In this extension, ambiguity is now over full latent tuples rather than single fact values: two tuples are confusable when the allowed partial observations fail to separate them. The induced graph is therefore a failure map for the architecture.

The graph can be handled either explicitly or implicitly. If there are d facts over an alphabet of size q, then the latent state space has cardinality q^d, so any explicit confusability graph materialization already starts from an exponentially large vertex set.(L: MFT113) A naive explicit construction ranges over at most q^{2d} ordered state pairs.(L: MFT116–117) For that reason the appropriate algorithmic interface is implicit: the formalization packages adjacency as a Boolean confusability oracle on state pairs, proves that this oracle is equivalent to the confusability relation itself, and also packages an equivalent agreement-set oracle together with a linear bound on the total view-membership work needed by the direct scan implementation. (L: MFT114–115, MFT132–135) Under the standard explicit representation

in which a latent tuple is stored as a length-d coordinate array and each admissible view is stored as its coordinate subset, this one-shot oracle is therefore computable by direct inspection in deterministic time $O(d+\sum_\ell |V_\ell|)$, hence $O(Ld)$ for L views: compute the agreement set of the two tuples once, then scan the views and test whether some view is contained in that agreement set. The theory below is therefore compatible with implicit graph access even when explicit graph materialization is impractical, while still leaving the one-shot adjacency test itself polynomial in the size of its explicit input.

The deterministic restriction is also not fundamental at the graph-construction level. The Lean development formalizes a support-overlap extension in which each location may emit any observation from a finite support set, and two states are confusable when some admissible location has overlapping observation support on those states. The present deterministic partial-view model embeds into that broader framework as the singleton-support special case, so the graph construction itself already extends beyond exact deterministic views even though the paper develops only the deterministic architectural arc.(L: MFT110–112)

At the same time, the exact current coordinate-subset model is *not* graph-universal on the full tuple space, and the induced graph class can be characterized more sharply than mere vertex transitivity. The formalization first shows that equality at a view is equivalent to containment of that view inside the coordinate-agreement set of the two tuples, so confusability depends only on which coordinates agree, not on the actual symbols carried there.(L: MFT118–119, MFT122) From any view family one therefore obtains an upward-closed family of coordinate subsets,

$$\mathcal{U} := \{S \subseteq [F] : \exists \ell \text{ with } V_\ell \subseteq S\},$$

such that two distinct tuples are adjacent if and only if their agreement set lies in $\mathcal{U}$. Conversely, every upward-closed family $\mathcal{U}$ arises from some coordinate-view architecture, so on the full labeled tuple space the realizable confusability relations are exactly the agreement-set relations cut out by upward-closed families.(L: MFT125–128) When the alphabet has at least two symbols, every proper coordinate subset occurs as the agreement set of some distinct pair of tuples, so this characterization is complete up to the irrelevant full-agreement case corresponding to identical tuples.(L: MFT129–131) Equivalently, coordinatewise alphabet permutations preserve the confusability relation and induce graph automorphisms; for any two latent tuples there exists such an automorphism carrying one to the other.(L: MFT120–124) Thus the exact model realizes a specific monotone agreement-set graph class rather than an arbitrary finite graph family. In particular, non-vertex-transitive graphs such as a three-vertex path cannot arise as full confusability graphs in this exact model. What remains open is a classification up to unlabeled graph isomorphism, and the corresponding classification for restricted realized-state subsets or for the stochastic support-overlap extension.

Theorem V.1 (Exact recovery as graph colorability)**.** *For a finite multi-fact latent tuple observed through a finite family of partial-coordinate views, exact zero-error recovery with a T-ary auxiliary tag exists if and only if the induced confusability graph is T-colorable.*

(L: MFT3–4, MFT20)

Proof. Let G_{conf} be the confusability graph.

Necessity. Suppose exact recovery with T tags is possible. If two latent tuples are adjacent in G_{conf}, then by definition some allowed observation leaves them indistinguishable. Exact recovery therefore cannot assign them the same tag, because the joint observation-tag pair would then coincide on two distinct latent tuples and Theorem IV.1 would be violated. Hence the tag assignment is a proper T-coloring of G_{conf}.

Sufficiency. Suppose c is a proper T-coloring of G_{conf}. Define the auxiliary tag of a latent tuple to be its color. If two latent tuples produce the same observation and the same color, then they are confusable and adjacent, contradicting proper colorability. Therefore the joint observation-color map is injective, and Theorem IV.1 yields an exact decoder. ■

An edge of G_{conf} records a specific pair of latent states that the view architecture cannot separate exactly, and a proper coloring is exactly a disambiguation budget that resolves those failures. Exact recovery cost is determined by failure topology, not by the count of sources alone.

This statement is packaged through an explicit simple confusability graph object, and for n repeated copies the full block confusability graph is identified with the n-fold strong power of the one-shot confusability graph. The zero-error criterion is therefore literally a graph-colorability theorem on an explicit power family rather than only an analogy.(L: MFT34–35)

Theorem V.2 (Success-set characterization)**.** *In the same model, exact decoding on a designated success set S is equivalent to T-colorability of the induced confusability subgraph on S.*

(L: MFT9)

Proof. Restrict the argument of Theorem V.1 to the success set. Exactness is required only on S, so the coloring condition is required only on confusable pairs inside S. ■

Theorem V.3 (Exact finite-error success budget)**.** *For each tag alphabet size $T > 0$, there is a largest success-set size*

$$M_T := \max\{|S| : S \text{ induces a } T\text{-colorable confusability subgraph}\}.$$

An exact decoder with T tags exists on a success set S if and only if $|S| \le M_T$ after replacing S by some T-colorable success set of size M_T. Under the uniform source, the optimal exact success probability is therefore $M_T/|\mathcal{X}|$.

(L: MFT14–15)

Proof. Theorem V.2 identifies exact success sets with induced T-colorable subgraphs. Since the latent alphabet is finite, a maximum-cardinality such set exists. The mechanized characterization exposes this optimum directly. In the binary four-cycle witness, the optimum at $T = 1$ is exactly two states, so the best uniform exact success probability is $2/4 = 1/2$. ■

The same characterization is also available in weighted form. For any finite source weight w on the latent alphabet,

the maximum mass of a T-colorable induced subgraph is the exact finite success value under budget T. This is the weighted version of the same structural claim: under limited budget, the best exact decoder is the largest colorable safe subworld allowed by the induced failure graph.(L: MFT18–19)

Equivalently, this optimum can be packaged as an explicit exact finite rate-distortion value for the extended model at tag budget T: every exact success set has weighted mass at most this value, and some exact decoder attains it. Thus the extended model no longer has only a converse inequality; at the one-shot level it has an exact weighted success law rather than merely an upper bound.(L: MFT23–24)

Theorem V.4 (First non-clique witness)**.** *There exists a binary two-fact system with single-coordinate observation views whose confusability graph is not a clique. In that system, two tags suffice for exact recovery, one tag is impossible, and any one-tag decoder can be exact on at most half of the four latent states under the uniform source.*

(L: MFT5–8, MFT10)

Proof. The witness is the four-state binary square with one location revealing the first coordinate and the other revealing the second. States that agree on one coordinate are confusable through the corresponding location, but opposite corners are not confusable, so the graph is a 4-cycle rather than a clique. An explicit exact two-tag decoder is obtained by the parity coloring $c(x_1, x_2) = x_1 \oplus x_2$. With one tag, any exact success set must be an independent set of the cycle, hence has size at most two, yielding success probability at most $1/2$ under the uniform source. ■

Theorem V.5 (Block-composition law)**.** *If two partial-observation systems are colorable with tag alphabets of sizes T_1 and T_2, then their block-composed product system is colorable with a tag alphabet of size T_1T_2.*

(L: MFT11–13, MFT16)

For later use, recall the strong product definition: if G and H are graphs, then $G \boxtimes H$ has vertex set $V(G) \times V(H)$, and (u, v) is adjacent to (u', v') exactly when either $u = u'$ and $v \sim v'$, or $u \sim u'$ and $v = v'$, or $u \sim u'$ and $v \sim v'$. The block-composition theorem is model-specific because the admissible product views generate exactly these three adjacency cases.

Proof. Take proper colorings of the two component confusability graphs and encode the pair of colors as one color in the product alphabet. If two product states are confusable, then some allowed product view fails to separate them. In the product system this means that either the first coordinates agree and the second coordinates are confusable, or the second coordinates agree and the first coordinates are confusable, or both coordinates are confusable. These are exactly the three adjacency cases in the strong product $G_1 \boxtimes G_2$. Hence every confusable product pair is separated by at least one component coloring, and therefore by the paired color as well. ■

This product law now has both directions in the formal development. The upper direction is the multiplicative coloring construction above. More sharply, once both location alphabets are nonempty, the product confusability graph is exactly the strong product of the two component confusability graphs. The graph object is therefore not just analogous to zero-error graph products; it is literally a strong-product construction for the induced failure map. (L: MFT25–26)

The lower direction is clique-based: if the first component contains a confusability clique of size c_1 and the second contains one of size c_2, then the product system contains a confusability clique of size c_1c_2, so any exact product decoder requires at least c_1c_2 tags. The same mechanism also propagates to finite-error success sets: the largest exactly decodable success set in the product system is at least the product of the largest exactly decodable success sets in the two components. At the one-shot zero-error coding level, the maximum independent-set code size also obeys the same product lower bound. Thus the extended model now has an honest strong-product graph law together with matching lower/upper budget consequences.(L: MFT17)

At this point the induced graph class has four explicit structural features. It contains non-clique graphs, as witnessed by the binary square. It contains the cluster-collapse subclass on which the later equality theorem is exact. It is closed under the block-composition operation, which becomes strong product on confusability graphs. And, by the agreement-set characterization above, it is exactly a monotone agreement-set class rather than an arbitrary graph family on the full tuple space. The iterated family below packages these features into a concrete infinite class.

a) Example family: iterated binary squares.: The four-state witness above is only the smallest member of a larger model-generated family. Let W denote the binary square system of Theorem V.4. Composing m independent copies of W yields a system with $2m$ binary fact coordinates and 4^m latent states whose confusability graph is the m-fold strong power

$$C_4^{\boxtimes m}.$$

Because each copy of W is exactly recoverable with two tags, repeated application of Theorem V.5 gives exact recovery on the m-fold product with 2^m tags. Conversely, each copy contains a confusability clique of size 2, so the clique lower bound in the same product theorem yields a confusability clique of size 2^m in the product system. Hence every exact decoder for this family requires at least 2^m tags, and therefore the exact tag budget is

$$T_{\text{exact}}(W^{\otimes m}) = 2^m.$$

This provides an infinite family of non-clique examples, generated directly by the model, on which the graph object, the exact budget, and the strong-product law all interact nontrivially rather than only in the single four-cycle case. At this point the paper has moved decisively beyond the trivial threshold story: above the unit-rate corner, not all failures are equally severe, because the architecture can induce a non-clique topology with a strictly more structured recovery law. (L: MFT11–13, MFT16–17, MFT29–30)

Once exact one-shot recovery is governed by graph colorability, repeated composition lifts the theory to strong graph powers and asymptotic zero-error rates. The next subsection makes that lift explicit.

VI. Block Composition, Strong Powers, and Asymptotic Capacity

This section studies how the induced failure topology scales. Once exact recovery is governed by colorability of the one-shot confusability graph, repeated composition does not reset the ambiguity; it composes the same failure structure across blocks. The next results identify the correct strong-power object and the resulting asymptotic zero-error rate.

A. Motivation: Repeated Composition

What happens if we repeat the partial-view experiment n times? If we compose n independent copies of the same encoding system, does the ambiguity explode uncontrollably, or does it grow at a predictable rate?

The answer depends on the failure topology:

- If the system were *fully* incoherent (confusability graph is a clique), every block would multiply the confusion. The ambiguity would grow without any exploitable structure.
- Because our confusability graph has *structure* (as in the 4-cycle example of Section V-A), the ambiguity growth is controlled by that structure. Some state pairs remain distinguishable even across multiple blocks.

Under n-fold block composition, the confusability graph becomes its n-fold *strong power*:

$$G_n \;=\; G^{\boxtimes n}.$$

Independent sets in the strong power relate systematically to independent sets in the base graph, yielding a supermultiplicative growth law that converges to a well-defined asymptotic rate. The theorem below quantifies this rate.

B. Block Composition Law

A genuine supermultiplicative block law holds:

$$\alpha_{m+n} \;\geq\; \alpha_m \alpha_n,$$

where α_n denotes the largest one-shot zero-error code size in the n-block composed system. This is the correct discrete precursor to Shannon-capacity analysis: the admissible code-size sequence is already supermultiplicative at the graph level. *(L: MFT22)*

More sharply, the block-level graph itself factors correctly. Confusability between concatenated $(m+n)$-block states is equivalent to adjacency in the strong product of the m-block and n-block confusability graphs, packaged formally as an explicit graph isomorphism. Thus the repeated-block system is not only bounded by strong-product arguments at the codebook level; it is literally a strong-product graph-power object for the same induced failure map. *(L: MFT29–30, MFT38–39)*

a) Conjunction, not disjunction.: Block confusability requires a *single location tuple* $\ell = (\ell_1, \ell_2)$ that works *simultaneously* for both blocks:

- Block 1: observe location ℓ_1, must match
- Block 2: observe location ℓ_2, must match

Two $(m+n)$-block states are confusable iff there exists one location tuple making observations match across *all* blocks. This is not "confusable in block 1 OR confusable in block 2" but rather "$\exists$ location tuple, $\forall$ blocks: match." That universal quantification over blocks is exactly the strong product condition: for each block, either equal or confusable (matching on some location), with at least one block strictly confusable.

b) Concrete example.: Consider two copies of the binary square. State $((0,0),(0,0))$ vs $((0,1),(1,1))$:

- Block 1: $(0,0) \sim (0,1)$ in C_4 (agree on x_1)
- Block 2: $(0,0) \not\sim (1,1)$ in C_4 (disagree on both coordinates)

An "OR-product" would mark these confusable because block 1 matches. But the actual model requires *one* location pair (ℓ_1, ℓ_2) matching *both* blocks. Block 1 can match via $\ell_1 = 1$ (observing x_1). Block 2 cannot match: location 1 gives 0 vs 1, location 2 gives 0 vs 1. No location pair works, so these states are not confusable, exactly as the strong product predicts.

Iterating the same construction yields the derived n-block lower bound. If α_1 denotes the one-shot maximum independent-set code size, then the mechanized block-power theorem gives

$$\alpha_1^n \;\leq\; \alpha_n,$$

where α_n is the largest one-shot zero-error code size in the n-fold block-composed system. This is the first asymptotic-capacity scaffold in the extended model: repeated block composition already forces the expected exponential growth law before any Shannon-capacity or ϑ-style upper theory is introduced. Once the architecture determines the one-shot failure graph, that same graph already controls the asymptotic lower law. *(L: MFT21)*

For notation, write $\Theta(G)$ for the Shannon capacity of a confusability graph G in logarithmic units, and write $\vartheta_\infty(G)$ for the corresponding asymptotic Lovász-ϑ upper value. When the object is a view family $\mathcal{V}$ rather than an abstract graph, we write $\Theta(G_\mathcal{V})$ and $\vartheta_\infty(G_\mathcal{V})$ for the same quantities computed on the induced confusability graph $G_\mathcal{V}$.

These normalized block rates can be packaged into an explicit lower asymptotic capacity envelope

$$C_* \;:=\; \sup_{n \geq 1} \frac{\log \alpha_n}{n},$$

implemented as an ENNReal supremum of the positive block-rate sequence. Formally, α_n is identified directly with the maximum finite independent-set size of the n-block confusability graph, the block graph is identified with the n-fold strong power of the one-shot confusability graph, and the same envelope is proved equal to the graph-side Shannon lower-capacity expression built from those strong powers. *(L: MFT31–32, MFT35–36)*

The asymptotic scaffold is also not limited to a lower envelope statement. On the graph side itself, strong-product lower bounds for independent-set cardinality induce monotonicity of normalized strong-power rates along repeated multiples. Specializing those generic graph theorems back to the model-generated confusability graph yields the repeated-block statement below. *(L: MFT40–42)*

Repeating a fixed m-block system k times yields

$$\alpha_m^k \;\leq\; \alpha_{km},$$

and hence

$$\frac{\log \alpha_m}{m} \leq \frac{\log \alpha_{km}}{km}.$$

Thus rates along repeated block multiples are monotone from below toward the same capacity envelope. This is still a lower theory rather than a full Shannon-capacity theorem, but it is already a genuine asymptotic graph-rate statement for the induced failure topology.(L: MFT33)

Once the block confusability graph has been identified with strong powers of the one-shot graph, the asymptotic rate theory is exactly the classical Shannon-capacity framework specialized back to the model-generated graph family.

Theorem VI.1 (Asymptotic Shannon-capacity theorem)**.** *Let α_n denote the largest one-shot zero-error code size in the n-block composed system. Then the normalized block rates*

$$\frac{\log \alpha_n}{n}$$

converge to a real asymptotic Shannon-capacity value, and that value is equal to the supremum of the finite block-rate sequence.

(L: MFT43–49, MFT56)

Proof. The argument rests on two observations: supermultiplicativity of independent-set sizes under strong product, and Fekete's lemma.

Supermultiplicativity. Recall that the n-block confusability graph G_n is the n-fold strong power $G^{\boxtimes n}$ of the one-shot graph G. For any two graphs H and K, an independent set in $H \boxtimes K$ can be constructed from independent sets in H and K as follows. If $I_H \subseteq V(H)$ and $I_K \subseteq V(K)$ are independent, then the Cartesian product $I_H \times I_K \subseteq V(H) \times V(K)$ is independent in $H \boxtimes K$: if (u, v) and (u', v') are adjacent in the strong product, then either $u = u'$ and $v \sim v'$, or $u \sim u'$ and $v = v'$, or $u \sim u'$ and $v \sim v'$. In each case at least one of u, u' or v, v' is an edge within the respective independent set, which is impossible. Hence $\alpha(H \boxtimes K) \geq \alpha(H)\alpha(K)$. Iterating gives $\alpha_{m+n} = \alpha(G^{\boxtimes(m+n)}) \geq \alpha(G^{\boxtimes m})\alpha(G^{\boxtimes n}) = \alpha_m \alpha_n$, so the sequence $\{\alpha_n\}$ is supermultiplicative.

Fekete convergence. The inequality $\alpha_{m+n} \geq \alpha_m \alpha_n$ is equivalent to subadditivity of the sequence $\{-\log \alpha_n\}$. The trivial bound $\alpha_n \leq |V(G)|^n$ ensures that $-\log \alpha_n \geq -n \log |V(G)|$, so the sequence is bounded below. By Fekete's subadditivity lemma, the normalized limit

$$\lim_{n\to\infty} \frac{\log \alpha_n}{n} = \sup_{n \geq 1} \frac{\log \alpha_n}{n}$$

exists and equals the supremum of the finite block-rate sequence. This limit is the asymptotic Shannon capacity $\Theta(G)$ in logarithmic units.

The mechanized development packages the same argument with explicit sequence objects and convergence lemmas; the textual expansion above makes the independent-set product construction and Fekete application explicit for accessibility. ■

c) Running example: Binary square.: For the 4-cycle C_4 from Section V-A, the independence number is $\alpha(C_4) = 2$ (choose either diagonal). The strong power $C_4^{\boxtimes n}$ has $\alpha(C_4^{\boxtimes n}) = 2^n$, giving normalized rate $(\log 2^n)/n = \log 2$. The asymptotic Shannon capacity is therefore $\Theta(C_4) = \log 2$. This matches the exact tag-budget growth from the iterated binary-square family discussed earlier: the one-shot budget is 2, and the m-block exact budget grows as 2^m.

d) Relation to graph capacity theory.: This places the extended model directly in the classical zero-error graph-capacity setting. The difference from the classical channel picture is generative rather than formal: here the confusability graphs arise from deterministic view families and partial observations on latent tuples. We do not claim a new general Shannon-capacity theorem for arbitrary graphs; the new content is that this partial-view model reaches the classical graph machinery in a non-clique regime, so the architecture determines the graph once and that graph then determines both one-shot and asymptotic recoverability. The clique-fiber regime recovers the classical cluster-graph equality case, while the transitivity criterion identifies when that case is produced by the view-family model. The non-clique witness already shows that the model also generates genuinely nontrivial graph behavior.

VII. Theta Upper Theory

This section supplies the asymptotic upper law for the same induced failure graph. The previous section showed that repeated composition turns view-induced ambiguity into strong powers and hence into an asymptotic recoverability rate. The question now is how large that rate can be. The first upper bound is complement-chromatic. The stronger package identifies the same one-shot upper invariant with standard orthonormal, primal-PSD, and dual-theta forms and then lifts those bounds to strong powers.

The complement-chromatic bound is the first upper bound. The complement of the strong-power confusability graph is identified with the coordinatewise "there exists an offending coordinate" graph on the complement, and a coloring of the one-shot complement graph lifts to a coloring of every strong power by coloring each coordinate separately. Consequently,

$$\alpha(G^{\boxtimes n}) \leq \chi(\overline{G})^n, \qquad \frac{\log \alpha(G^{\boxtimes n})}{n} \leq \log \chi(\overline{G}),$$

and the asymptotic capacity is bounded above by $\log \chi(\overline{G})$.(L: MFT50–55)

At the one-shot level, the same upper invariant is matched with standard witness, orthonormal-representation, primal-PSD, and dual-theta formulations by explicit feasible-object equivalences. Applying these one-shot bounds to strong powers yields subadditive upper-rate sequences, so Fekete's lemma gives asymptotic primal and dual upper values, each equal to the infimum of its finite power-rate bounds. The finite block rates are bounded above by either sequence, hence

$$\Theta(G) \leq \vartheta_\infty(G).$$

The same induced topology that governs exact recovery therefore also carries a classical asymptotic upper theory. The

formal development proves these one-shot equivalences and the resulting asymptotic primal/dual upper laws.(L: MFT57–84)

Interpretationally, the complement-chromatic bound is the first combinatorial ceiling, while the Lovász-ϑ package supplies the sharper geometric and semidefinite upper theory attached to the same confusability graph. Once a deterministic partial-view architecture generates a failure graph, the standard zero-error upper hierarchy attaches to that graph without further model-specific modification.

What remains open is the sharpness of this upper theory for the graph classes generated by the extended model, not the existence of standard primal–dual theta packaging.

VIII. Equality Characterization

This section asks when the upper theory of the induced failure graph is exactly sharp. The graph-theoretic equality facts used here are classical for cluster graphs; the model-specific question is when the partial-view architecture generates that collapse regime. Transitivity of confusability is the key structural condition exporting the view-family model to the cluster-graph equality case.

Open question. A precise open problem is to characterize which upward-closed agreement-set families produce transitive confusability, and therefore trigger the cluster-graph equality collapse. We have identified boundary facts: the binary square is a small witness where transitivity fails, while trivial single-view families and the fiber-coherent/meet-witnessed families provably produce transitive confusability and hence the equality collapse. A complete classification of upward-closed families that yield transitivity remains open and is an attractive direction for further work.

The original clique-fiber regime is now isolated as an equality subclass rather than only a threshold corollary. If a graph is generated by a surjective label map $x \mapsto b(x)$ with adjacency exactly between distinct vertices in the same fiber, then one representative from each fiber forms an independent set of size $|\mathcal{B}|$, while the complement graph is colorable by the same label map using $|\mathcal{B}|$ colors. This equality for cluster-graph structure is classical; the mechanized development packages the model-specific export needed here into a restricted-class theorem:

$$C(G) = \vartheta_\infty(G) = \log|\mathcal{B}|.$$

Thus the extended theory now contains both a genuine non-clique graph regime with strict upper/lower separation and a clean equality regime recovering the original fiber picture inside the same asymptotic Lovász-ϑ framework. In particular, transitivity of confusability exports the model-generated graph directly to the classical cluster-graph setting through the following mechanized implication:

$$\text{confusability is transitive} \Longrightarrow G_{\text{conf}} = \text{Cluster}(\Pi_{\text{cc}}).$$

Hence, if confusability is transitive, the base confusability graph is exactly the cluster graph on connected components, and the model-specific equality theorem yields

$$\Theta(G_{\mathcal{V}}) = \vartheta_\infty(G_{\mathcal{V}}) = \log|\Pi_{\text{cc}}|,$$

where Π_{cc} is the connected-component partition of the base confusability graph. Between arbitrary base models and full fiber coherence, a checkable sufficient condition is that the view family be *meet-witnessed*: every pair of allowed views contains another allowed view inside their intersection. Under that condition, any two confusability witnesses can be pulled back to a common subview, so confusability is transitive and the same equality follows. Fiber coherence is then a stronger sufficient structural condition: if equality at one allowed location already fixes the full observation transcript, transitivity follows, the connected components are exactly the realized transcript fibers, and the equality sharpens further to

$$\Theta(G_{\mathcal{V}}) = \vartheta_\infty(G_{\mathcal{V}}) = \log|\mathcal{T}_{\text{real}}|.$$

At the witness level, this same collapse admits a local composition form: base confusability is the edge union of the single-view cluster graphs, and transitivity is equivalent to closure of those fixed-view witnesses under two-step composition through an intermediate state. This is the exact local criterion for the current equality mechanism: the collapse to a cluster graph is determined by how single-view witnesses compose, not by an opaque global coincidence of the full confusability graph. Thus the equality question is structural: it identifies when genuinely non-clique failure geometry collapses to exact cluster behavior.(L: MFT85–91, MFT100–102, MFT107–109, MFT136–137)

a) Running example: Binary square.: The binary square of Section V-A does *not* have transitive confusability: $(0,0) \sim (0,1)$ and $(0,1) \sim (1,1)$, but $(0,0) \not\sim (1,1)$. The 4-cycle is not a cluster graph, so the transitivity-based equality collapse does not apply. In this special case the classical graph invariants still satisfy $\Theta(C_4) = \vartheta_\infty(C_4) = \log 2$, but that coincidence no longer comes from the cluster-collapse mechanism isolated in this section. The binary square therefore marks the boundary of the paper's equality route: it is the first non-clique witness, yet not a transitive one.

Hierarchy summary. This equality route has one transitivity-based sufficient condition and two stronger sufficient conditions. The table below records only the logical structure and the resulting capacity value.

Condition	Logical status	Structural consequence	Capacity value
Fiber coherence	Stronger sufficient condition	Connected components are realized transcript fibers	$\log\lvert\mathcal{T}_{\text{real}}\rvert$
Meet-witness	Sufficient for transitivity and collapse	Base confusability collapses to the component cluster graph	$\log\lvert\Pi_{\text{cc}}\rvert$
Transitivity	Sufficient collapse condition used here	$G_{\text{conf}} = \text{Cluster}(\Pi_{\text{cc}})$	$\log\lvert\Pi_{\text{cc}}\rvert$

IX. Affine Dual: Coordinate Matroid

The preceding sections developed the main graph-capacity and equality arc for partial views. This section turns to a second lens on the same model: which fact coordinates

determine others across the realized state family? Under an affine restriction on the realized state family, that determination problem reduces to standard linear algebra and yields a representable matroid on fact indices. This matroid is the coordinate-side dual of the same structure that generates the confusability graph, and it provides computationally tractable upper bounds on confusability and capacity. This matroid perspective connects the problem to the rich literature on representable matroids in coding theory and combinatorial optimization, where matroid rank functions provide tractable bounds on code parameters.

Proposition IX.1 (Affine fact-matroid specialization)**.** *Assume the realized state family is affine over a field, so that the valid latent tuples form an affine translate of a linear subspace of the ambient fact space. For any fact set S and fact index i, semantic determination of i by S is equivalent to membership of the coordinate functional for i in the linear span of the coordinate functionals indexed by S. Consequently the fact indices carry a representable matroid: a fact set is independent exactly when its coordinate functionals are linearly independent, and the minimal determining fact sets are exactly the bases, all of common cardinality equal to the rank.*(L: AFM1–5)

Proof. Let the affine realized-state family be $A = a_0 + V$, where a_0 is a fixed origin and V is a linear subspace of the ambient fact space. Fix a fact set S and a fact index i.

By definition, fact i is semantically determined by S exactly when for all $x, x' \in A$, agreement on every coordinate in S forces agreement on coordinate i. Writing $d = x - x'$, this is equivalent to the statement that every direction vector $d \in V$ whose coordinates in S vanish also satisfies $d_i = 0$.

Let e_j denote the coordinate functional for fact j. The condition above says precisely that

$$V \cap \bigcap_{j \in S} \ker(e_j) \subseteq \ker(e_i).$$

In finite-dimensional linear algebra, this is equivalent to e_i lying in the span of the coordinate functionals $\{e_j : j \in S\}$. Thus semantic determination by S is exactly span membership of the corresponding coordinate functional.

The coordinate functionals therefore realize a representable matroid on fact indices. In that matroid, independence is linear independence of the coordinate functionals, bases are the maximal independent spanning sets, and every basis has the common rank cardinality. Since span corresponds exactly to semantic determination, the minimal determining fact sets are precisely those bases. ■

The matroid and the confusability graph are complementary objects induced by the same realized-state model. Under the common specialization to finite affine families with coordinate-projection views, the matroid rank provides *computationally tractable* upper bounds on confusability and capacity. The tractability claim is representation-sensitive and should be read that way. We assume the affine family $A = a_0 + V$ is given by an explicit linear presentation of V, for example a basis or generator matrix over $\mathbb{F}_q$, together with the explicit coordinate-subset view family. Under that input model, for each coordinate set S the quantity $t(S)$ is exactly the finite-dimensional rank of the restricted coordinate map $\pi_S|_V$: the formalization identifies the span-based rank quantity with the finrank of the image of that explicit projection map.(L: AFM10–11) It also proves the basic size bounds $t(S) \le |S|$ and $t(S) \le \dim V$, equivalently $\dim(\mathrm{im}(\pi_S|_V)) \le \min\{|S|, \dim V\}$, and shows that $t(S) = |S|$ on linearly independent coordinate families while $t(S)$ reaches the full ambient determining rank exactly on determining sets.(L: AFM6–9, AFM12) Hence each $t(S)$ is a standard linear-algebraic rank quantity that can be computed by Gaussian elimination on an explicit presentation of V, and the bounds of Corollary IX.3 are then computable in polynomial time in the ambient dimension, the dimension of V, and the number of admissible views. Without an explicit linear presentation of V, no algorithmic tractability claim is intended. The utility of this matroid structure is immediate: $t(S)$ provides an explicit upper bound on the Shannon capacity of the induced confusability graph, bypassing the hardness of the general capacity computation. Computing Shannon capacity for general graphs is hard (the independence number is NP-hard, and even the Lovász-ϑ bound requires semidefinite programming). In contrast, once the affine family is explicitly presented, the relevant rank quantities reduce to standard linear algebra.

A. View-fiber dimensions and capacity

Let $A = a_0 + V$ with V an r-dimensional linear subspace over a finite field $\mathbb{F}_q$, and let admissible views be coordinate-subset projections. For a coordinate set S write $t(S)$ for the rank of the corresponding coordinate functionals on V.

Proposition IX.2 (View-fiber clique sizes)**.** *Let A and $t(S)$ be as above. Each projection-fiber for view S is an affine subspace of dimension $r - t(S)$ and size $q^{r-t(S)}$. The single-view confusability graph G_S is therefore a disjoint union of $q^{t(S)}$ cliques each of size $q^{r-t(S)}$. In particular*

$$\alpha(G_S) = q^{t(S)}, \qquad \Theta(G_S) = t(S) \log q,$$

where α is the independence number and Θ is Shannon capacity (logarithms in the same base used elsewhere).

Proof. The projection onto coordinates S is an affine-linear map whose linear part restricted to V has rank $t(S)$. Its kernel is therefore a linear subspace of V of dimension $r - t(S)$, so every fiber is an affine translate of that kernel and has cardinality $q^{r-t(S)}$. The fibers partition A, and within each fiber every pair of states is confusable under S, so G_S is a disjoint union of equal-size cliques. The number of fibers equals $|A|/q^{r-t(S)} = q^{t(S)}$, so one can select one representative per fiber to obtain an independent set of size $q^{t(S)}$, whence $\alpha(G_S) = q^{t(S)}$. The block-power identity for such clustered graphs gives $\alpha(G_S^{\boxtimes n}) = q^{nt(S)}$, and normalization yields $\Theta(G_S) = t(S) \log q$. ■

Corollary IX.3 (Matroid ranks give capacity bounds)**.** *Let G be the full confusability graph generated by a family of coordinate-subset views $\mathcal{S}$. Then*

$$\alpha(G) \le \min_{S \in \mathcal{S}} q^{t(S)} \qquad \textit{and} \qquad \Theta(G) \le \min_{S \in \mathcal{S}} t(S) \log q.$$

Thus the matroid rank function $t(\cdot)$ on coordinate sets supplies explicit upper bounds on one-shot and asymptotic exact-recovery rates for the induced confusability graph.

(L: AFM6–12)

Proof. An independent set for G must be independent in each single-view graph G_S, hence its size is at most $\alpha(G_S) = q^{t(S)}$ for every $S \in \mathcal{S}$. The inequalities follow from taking the minimum over $\mathcal{S}$ and applying the block-power argument from the proposition. ■

Remark. The matroid rank $t(S)$ is the "informational dimension" captured by coordinates in S, directly analogous to the rank function in representable matroids studied in combinatorial optimization [10] and coding theory [6]. Here $t(S) = r$ iff S determines the entire state, and any admissible view containing a basis removes confusability entirely (the induced graph is edgeless). These finite-affine, coordinate-view statements show the matroid is directly applicable to the graph/capacity arc; extensions beyond this specialization are directions for future work.

a) Running example: Binary square.: The binary square $\{0,1\}^2$ from Section V-A is an affine space over $\mathbb{F}_2$ with $r = 2$. Each single-coordinate view has matroid rank $t(\{1\}) = t(\{2\}) = 1$, giving fiber size $2^{2-1} = 2$ and single-view capacity bound $\Theta(G_{\{i\}}) \leq 1 \cdot \log 2 = \log 2$. The full confusability graph (the 4-cycle) achieves this bound, so the matroid rank bound is tight in this case. The matroid viewpoint shows that the 4-cycle structure arises precisely because each view captures only half the informational dimension.

X. Unit-Rate Structural Interpretation

The main graph-capacity arc studies what happens once ambiguity survives and acquires nontrivial topology. This section returns to the opposite boundary case of the same model: the unit-rate corner in which that ambiguity never appears. It records two direct interpretations of that boundary case: derivation realizes the unit-rate regime, and leaving it produces the manual-update gap proved later in Section XI.

A. Derivation as the Single-Source Structure

An encoding system has *unit independent rate* when $\mathrm{DOF}(C, F) = 1$. By Proposition III.6, this is exactly the regime in which every reachable state remains coherent. By Proposition III.7, any rate above 1 admits reachable disagreement states. Thus unit rate is the unique exact-consistency boundary, and the later rate-complexity statements are its operational cost interpretation.*(L: COH1–2, BND1, RAT1, RED1)*

B. Reading the Converse at Unit Rate

When ambiguity classes are trivial, no auxiliary information is needed and unit rate suffices. When a nontrivial ambiguity class survives the observation transcript, Theorem IV.3 shows that exact recovery requires a sufficiently large side-information alphabet. If that side information is unavailable but exact correctness is still demanded, additional independently accessible support is required. That is exactly what moves the system above unit rate and leads to the update-cost lower bound of Section XI. Derivation is therefore the dependence structure that preserves visible views while removing independent rate.*(L: CIA3, BND2, DER2)*

C. Rate and Manual Update Cost

An encoding may expose many visible copies of a fact, but only the independent copies govern manual synchronization cost. At unit rate, one manual update changes the authoritative source and derived views follow automatically; above unit rate, each independent location must be synchronized manually. This is the cost interpretation of the threshold.*(L: BND1–2)*

D. Derived Views

Definition X.1 (Derivation)**.** Location L_{derived} is *derived from* L_{source} for fact F iff an update to L_{source} determines the value of L_{derived} without an additional manual edit.

Derivation may occur at creation time, build time, commit time, or query time depending on the host system. The abstract point is unchanged: a derived location does not contribute an additional independently writable value for the same fact.

Proposition X.2 (Derivation Preserves Coherence)**.** *If L_{derived} is derived from L_{source}, then L_{derived} cannot diverge from L_{source} and does not contribute to DOF.*

(L: DER2)

Proof. By Definition X.1, the value at L_{derived} is determined by the value at L_{source} under admissible updates. There is therefore no reachable state in which the two locations carry incompatible values for the same fact. Since the derived location has no independently writable contribution, it does not increase DOF. ■

If all encodings of F except one are derived from that one, then Proposition X.2 leaves exactly one independent source, hence $\mathrm{DOF}(C, F) = 1$ and exact consistency follows from Proposition III.6.*(L: DER2, COH1)*

The same dependence structure can be realized in many host systems. Its role here is only to prepare the realizability criterion.

XI. Threshold and Rate-Complexity Corollaries

The main graph-capacity arc characterizes the structure of exact failure under partial views. This section records one downstream operational consequence at the opposite boundary: independent rate governs manual synchronization cost in the same finite deterministic model.

A. Cost Model

Definition XI.1 (Modification Cost Model)**.** Let δ_F be a modification to fact F in encoding system C. The *effective modification complexity* $M_{\text{effective}}(C, \delta_F)$ is the number of locations that must be edited manually to preserve correctness after applying δ_F.

Only manual edits are counted. Derived locations updated automatically by the host system contribute zero effective cost.

B. Upper Bound at Unit Independent Rate

Theorem XI.2 (Rate-1 Upper Bound)**.** *If DOF$(C, F) = 1$, then*

$$M_{\text{effective}}(C, \delta_F) = O(1).$$

(L: BND1)

Proof. When DOF$(C, F) = 1$, there is exactly one independently writable source location for F. Updating that location is one manual edit; every other representation of F is derived and is updated automatically. The number of manual edits therefore stays bounded by a constant independent of the number of visible encodings. ■

C. Lower Bound Above the Threshold

Theorem XI.3 (Above-Threshold Lower Bound)**.** *If fact F is encoded at n independent locations, then*

$$M_{\text{effective}}(C, \delta_F) = \Omega(n).$$

(L: BND2)

Proof. Each independent location can retain an outdated value unless it is edited directly. After a source update, every one of the n independent locations can therefore witness a distinct stale copy of the same fact. Any exact repair strategy must bring each such location back into agreement, and in the worst case no single manual action can repair two independently writable locations at once. Hence the effective modification complexity grows at least linearly in n. ■

Lemma XI.4 (Information-Constrained DOF Lower Bound)**.** *If the available side-information mechanism exposes at most 2^L distinguishable tags while the latent ambiguity class has size $K > 2^L$, then no zero-error resolver can identify the true state from those tags alone. Any architecture that still guarantees exact correctness must therefore rely on additional independently accessible discriminating support, moving the system above unit rate.*

(L: CIA4)

Proof. Theorem IV.3 rules out exact recovery from the available tag budget. If exact correctness is nevertheless required, the architecture must supplement the insufficient side-information channel with additional independently accessible information. In the present model, that means leaving the rate-1 regime and paying the synchronization cost of Theorem XI.3. ■

D. The Unbounded Gap

Theorem XI.5 (Unbounded Gap)**.** *The ratio between the modification cost of an architecture with n independent encodings and the modification cost of a rate-1 architecture diverges:*

$$\lim_{n\to\infty} \frac{M_{\text{DOF}>1}(n)}{M_{\text{DOF}=1}} = \infty.$$

(L: BND3)

Proof. By Theorem XI.2, a rate-1 architecture has constant effective modification complexity. By Theorem XI.3, an architecture with n independent encodings incurs linear cost in n. The ratio therefore grows without bound. ■

Once a fact is duplicated across many independently writable locations, the cost of preserving exact consistency scales with the number of independent copies, not with the visibility of the fact.

XII. Operational Realizability Criterion

The main graph-capacity arc characterizes what happens once ambiguity survives and must be resolved structurally. This section turns to the opposite boundary case of the same model: when can a concrete host system realize the unit-rate corner isolated earlier, and what two structural properties are needed for that purpose? In integrity language, the question is when structural integrity can be both enforced and certified by the host itself. The resulting criterion is architectural rather than a new graph-capacity theorem.

A host system realizes the rate-1 regime when one location is authoritative and every secondary representation is maintained as a deterministic view. Two capabilities are required:

1) **Causal update propagation**: updates to the authoritative source must automatically propagate to derived locations.
2) **Provenance observability**: the system must expose enough structural information to verify which locations are authoritative and which are derived.

Definition XII.1 (Verifiable Structural Integrity)**.** An encoding system has *verifiable structural integrity* for structural facts if it both realizes structural integrity in reachable states and exposes enough host-native information to certify that this integrity-preserving dependence structure holds.

A. Confusability and Side Information

To connect realizability to information available at verification time, fix a fact F and consider the alternatives that remain compatible with the available observations. These alternatives form the same zero-error obstruction analyzed in Sections IV–VIII: if the host cannot separate the surviving ambiguity class, exact verification requires additional structural side information.

Lemma XII.2 (Confusability Clique Bound)**.** *If the confusability graph for F contains a clique of size k, then any zero-error side-information scheme distinguishing those k alternatives requires at least k distinct tags, equivalently at least $\log_2 k$ bits of side information.*

(L: OBS5)

Proof. Within a k-clique, every pair of alternatives is confusable under the base observation. A zero-error tag assignment must therefore separate all k states. Distinct states cannot share a tag, so at least k tags are required. ■

If a host system does not expose enough structural information to separate the remaining alternatives, exact verification

of the rate-1 regime is impossible. In security-adjacent terms, the host cannot certify that an apparently coherent state is genuinely authoritative rather than an undetected stale coincidence.

B. The Structural Timing Constraint

Definition XII.3 (Structural Fact)**.** A fact F is *structural* if every location encoding F is fixed when the underlying object, declaration, or artifact is created. After that moment, the encoding can be read and compared, but not retroactively re-created without a new edit event.

Structural facts are the canonical exact-consistency setting because they expose a clear creation moment. Examples include schema declarations, registry membership rules, dependency metadata, and interface-level constraints.

Theorem XII.4 (Timing Constraint for Structural Derivation)**.** *For structural facts, derivation must occur no later than the moment at which the relevant encoding locations become fixed.*

(L: REQ1, TRI1)

Proof. Let t_{fix} denote the time at which the structural encoding becomes fixed. A derivation performed strictly before t_{fix} cannot depend on the completed source value; a derivation performed strictly after t_{fix} cannot alter the already fixed structural representation. Therefore structural derivation must occur at t_{fix} or as part of the same creation/update event. ■

C. Requirement 1: Causal Update Propagation

Definition XII.5 (Causal Update Propagation)**.** An encoding system has *causal update propagation* if every update to a source location is followed by updates to all locations derived from that source with no reachable intermediate state in which source and derived locations disagree. Equivalently, propagation is part of the same admissible update event for the purposes of reachable-state analysis, rather than an asynchronous repair step with observable stale states.

This is the host-system manifestation of deterministic dependence. Without it, a temporal gap appears between source modification and derived-view repair.

Theorem XII.6 (Causal Propagation is Necessary for Unit Rate)**.** *Achieving independent rate* 1 *for structural facts requires causal update propagation.*

(L: REQ2)

Proof. By Theorem XII.4, structural derivation must occur at the moment the source-side structural encoding is fixed. If causal propagation is absent, then some derived location remains stale until a later manual action. During that interval the source and derived locations disagree, so the architecture does not realize exact consistency. Hence a verifiable rate-1 realization for structural facts requires causal propagation. ■

D. Requirement 2: Provenance Observability

Definition XII.7 (Provenance Observability)**.** An encoding system has *provenance observability* if it supports queries that reveal which locations encode a fact, which of those locations are authoritative, and which are derived.

Provenance observability is the structural side information needed to certify that the system is genuinely in the single-source regime rather than merely appearing coherent on a small sample of states.

Theorem XII.8 (Provenance Observability is Necessary for Verifiable Unit Rate)**.** *Verifying that independent rate* 1 *holds requires provenance observability.*

(L: REQ3)

Proof. To verify independent rate 1, one must enumerate the locations encoding the fact, identify the authoritative source, and confirm that every remaining location is derived from it. Without provenance queries, these checks cannot be completed from inside the host system. The architecture may be coherent on observed states, but the single-source claim is not verifiable. ■

E. Independence of the Two Requirements

The two requirements are logically separate.

Theorem XII.9 (Requirements are Independent)**.**
1) *An encoding system can have causal propagation without provenance observability.*
2) *An encoding system can have provenance observability without causal propagation.*

(L: IND1–2)

Proof. For (1), consider a system that automatically refreshes all derived views after each source update but exposes no query interface for its derivation graph. Coherence may hold operationally, but the single-source claim is not inspectable. For (2), consider a system that records complete provenance metadata yet requires users to trigger propagation manually after each change. The derivation structure is visible, but stale views remain reachable. Hence neither property implies the other. ■

F. The Realizability Theorem

Corollary XII.10 (Operational Realizability Criterion)**.** *An encoding system can achieve verifiable independent rate* 1*, equivalently verifiable structural integrity, for structural facts if and only if it provides both causal update propagation and provenance observability.*

(L: SOT1)

Proof. Necessity follows from Theorems XII.6 and XII.8. For sufficiency, assume both properties hold. Causal propagation ensures that every derived location is updated as part of the same structural event as the source; provenance observability then certifies that all secondary encodings are in fact derived from that source. The architecture therefore realizes

a verifiable single-source encoding, i.e., independent rate 1, which is exactly verifiable structural integrity in the sense of Definition XII.1. ■

Applied to representative hosts, this criterion separates systems in which propagation and provenance are both host-native from systems missing one or both capabilities. Appendix A collects those representative readings in one table, and Supplement A contains the companion case-study traces and measurement notes.

XIII. Related Work

A. Zero-Error and Side-Information Lineage

Shannon's zero-error framework and its graph-theoretic refinements by Körner and Lovász provide the closest conceptual starting point [1]–[3]. Witsenhausen's zero-error side-information problem is directly relevant because it studies exact recovery under side information through graph coloring and label budgets [4]. Our one-shot coloring statements lie in that lineage. As in Witsenhausen's formulation, an observation law induces the relevant confusability graph. The difference is that here the observation law is generated by a deterministic multi-fact tuple-space architecture with admissible coordinate-subset views, so the model itself constrains what graphs can arise.

a) Key distinction from Witsenhausen.: Both settings can be viewed as starting from an observation law and passing to the induced confusability graph, so there is no sharp "induced" versus "given" divide. The difference is architectural and structural. Witsenhausen's setting starts from a side-information law and studies the coding problem for the graph it induces. Our setting starts from a deterministic partial-view architecture on latent tuples, with observations obtained by admissible coordinate-subset projections, and asks what graph structure that architecture can generate and what recovery laws follow from it. This makes two model-side issues central: how the admissible view family constrains the induced graph class, and how changing the view topology changes the resulting failure graph and recovery budget. In the exact full-tuple-space coordinate-view model analyzed here, that graph class can now be characterized exactly: the realizable confusability relations are precisely those determined by upward-closed families of coordinate-agreement sets. The contribution is therefore not a new coloring theorem for arbitrary graphs, but an architectural realization theory connecting multi-fact tuple-space views, a fully characterized model-generated graph class, and exact zero-error recovery.

Slepian–Wolf coding and classical entropy converses supply the second line of influence [5], [6], [8], [9]. Coding-for-computing, functional compression, and graph-entropy/characteristic-graph viewpoints are also close neighbors because they study deterministic decoder targets and coding questions once an underlying graph or function structure has been specified [2], [11], [12].

Characteristic-graph vs. coordinate-projection models. Characteristic-graph and functional-compression formalisms handle arbitrary deterministic functions of the source and therefore subsume a broader class of recovery tasks than the present model. Our model intentionally restricts attention to coordinate-projection observations on labeled tuple spaces rather than arbitrary functions. That restriction is the source of our structural leverage: it allows a reduction of confusability to agreement-set membership and thereby enables proofs that (a) realizable confusability relations are exactly those determined by upward-closed agreement families, (b) coordinatewise alphabet permutations act by graph automorphisms, and (c) the induced graph class is monotone and closed under the block-composition (strong-product) operation. These specific structural consequences do not follow from the general characteristic-graph viewpoint and are the reasons the paper can give deductive, architecture-first results rather than only applying general functional-compression theorems.

In our setting the side information is deterministic rather than stochastic, but it plays the same operational role: exact recovery becomes impossible when the available observation/-tag alphabet is too small to separate the remaining alternatives.

After a confusability graph is fixed, the colorability, strong-power, Shannon-capacity, and Lovász-ϑ machinery used later is classical. In particular, capacity–ϑ equality for cluster graphs is classical; the novelty here is identifying transitivity of view-generated confusability as the model-side condition that produces this equality subclass, together with meet-witnessing and fiber coherence as structural sufficient conditions. The paper therefore does not claim a new theorem about arbitrary graphs, but a deterministic partial-view model that generates a fully characterized monotone agreement-set graph class on the full labeled tuple space, an exact recovery/success-set/weighted-success theory for that model-generated class, and a structural equality route inside it. What it does not attempt is the broader unlabeled-isomorphism classification, or the corresponding classifications for restricted realized-state families and stochastic support-overlap models.

B. Consistency-Constrained Storage and Interaction

Consistency-constrained storage problems, including multi-version coding and related distributed-storage converses, show that correctness requirements impose unavoidable information costs [13]. Here the object under study is a modifiable encoding architecture, and the main question is the exact-consistency cost of multiple independently writable representations of one latent fact.

C. Computational Realizations

Reflection, metadata systems, and maintenance mechanisms in host platforms provide examples of how the realizability conditions can be instantiated in practice [14], [15]. These examples are auxiliary to the zero-error theory rather than part of its main novelty claim.

D. Formal Verification

The Lean 4 formalization places the work in the tradition of mechanized mathematics and verified theory development [7], [16].

XIV. Conclusion

The paper's main contribution is a structural theory of exact failure under deterministic partial views. Admissible view families induce confusability graphs on latent states; those graphs record which latent alternatives the architecture cannot separate, and they govern the exact recovery law. In the exact full-tuple-space coordinate-view model, this graph class is characterized exactly by upward-closed agreement-set families rather than treated as an arbitrary graph input. In the multi-fact partial-observation extension, exact recovery becomes colorability, exactness on a success set becomes induced-subgraph colorability, and the exact finite weighted-success value is determined by the largest colorable induced subgraph.(L: MFT9, MFT18–24, MFT20, MFT125–131)

Repeated composition preserves that failure structure rather than erasing it: the block-rate sequence is supermultiplicative, its normalized rates converge by Fekete's lemma to a real asymptotic Shannon capacity equal to the supremum of the finite block rates, and the same capacity is bounded above by complement-chromatic and fixed Lovász-ϑ upper objects. The one-shot upper invariant is matched with standard orthonormal-representation, primal-PSD, and dual-theta forms. (L: MFT29–30, MFT43–84)

The upper theory is sharp on a structurally characterized subclass. For the original clique-fiber subclass, asymptotic Shannon capacity, the fixed Lovász-ϑ upper, and the logarithm of the number of fibers coincide. The same equality exports back to the original view-family model through a broader structural route: transitivity of confusability yields the component-cluster collapse, meet-witnessing is a sufficient condition for that transitivity, and fiber coherence is a stronger sufficient condition under which the connected components are exactly the realized transcript fibers.(L: MFT85–91, MFT100–102, MFT136–137)

The same latent-state framework also yields a fact-side affine specialization. The observation-side question asks which latent states are distinguishable from partial views; the fact-side question asks which coordinates determine others across the realized-state family. When the valid states form an affine family, the latter question becomes span membership of coordinate functionals, so the fact indices carry a representable matroid and the minimal determining fact sets are exactly the bases. In this affine regime, the same model carries a graph invariant on states and a matroid invariant on coordinates, and the latter supplies tractable upper bounds for the former. More sharply, the relevant affine rank quantity is exactly the finrank of the image of the restricted coordinate projection, which closes the representation-level link between the matroid language and explicit linear algebra.(L: AFM1–14)

The deterministic finite converse remains the foundation of this theory. Once the observation transcript leaves a nontrivial ambiguity class, the same obstruction can be stated as confusability, counting, conditional-entropy, decoder-output, and finite-gap constraints, together with deterministic data processing and a budgeted finite-error extension. The paper does not claim a new general theorem about arbitrary confusability graphs; rather, it shows that a partial-view encoding model can induce genuinely non-clique failure geometry and reach the classical graph-capacity machinery there. The same model also has a clean boundary theory: rate 1 is exactly the structural-integrity regime, higher rate makes integrity violations reachable, and realizability plus provenance characterize when that integrity can be certified by the host.

As downstream consequences of that foundation, unit rate is the unique zero-incoherence regime, any higher independent rate makes ambiguity reachable, and the same obstruction yields an $O(1)$ versus $\Omega(n)$ manual-update gap together with a realizability criterion based on causal propagation and provenance observability. Appendix A records representative host-level readings of that boundary-case criterion, and Supplement A adds before/after traces and case-study details showing how the same structural split appears in concrete systems.

This boundary-case classification also has an integrity reading. In Pattern A and Pattern B architectures, the host cannot reliably distinguish a valid derived state from an undetected stale state by its own structural interface. The resulting incoherence is an integrity exposure that leaves stale but plausible states reachable and, in adversarial settings, creates latent attack surface inside the architecture itself.

Computational representation. The graph-theoretic formulation is structurally exponential in the number of facts: for d facts over an alphabet of size q, the latent state space has cardinality q^d, so any explicit confusability graph materialization necessarily starts from an exponentially large vertex set.(L: MFT113) A naive explicit construction ranges over at most q^{2d} ordered state pairs, formalized in the artifact through the corresponding product-cardinality and pair-count bounds.(L: MFT116–117) At the same time, the graph need not be materialized: adjacency is already packaged by an implicit oracle on state pairs, formalized as a Boolean confusability test equivalent to the confusability relation itself, and also by an equivalent agreement-set oracle whose direct scan cost is bounded linearly in the total view size.(L: MFT114–115, MFT132–135) Under the explicit coordinate-view representation used throughout the paper, that one-shot oracle is therefore polynomial in its direct input size, computable by scanning the coordinates and the admissible view family in $O(d + \sum_\ell |V_\ell|)$ time, hence $O(Ld)$. In the affine specialization, the upper bounds are likewise polynomial-time computable once the direction space is presented explicitly by a basis or generator matrix, because the formalized rank quantity is exactly the finrank of the image of the restricted coordinate projection, so each bound reduces to Gaussian elimination on that explicit map.(L: AFM10–11) The exponential barrier therefore lies in full graph materialization and global capacity computation, not in the local adjacency test or in the affine upper-bound surrogate.

Limitations and future work. The present paper develops only the finite deterministic architectural arc. For the exact tuple-space coordinate-view model, the full labeled-tuple-space graph class is now characterized exactly by monotone agreement-set families; what remains open is the corresponding classification up to unlabeled graph isomorphism, the restricted-state and stochastic-support variants, and the sharpness of the upper theory on those broader subclasses.

A. Artifacts

The Lean 4 formalization is included as supplementary material. Appendix A records representative realizability readings, and Supplement A contains the companion case-study material.

Acknowledgment: AI-use Disclosure

Generative AI tools (including Codex, Claude Code, Augment, Kilo, and OpenCode) were used throughout this manuscript, across all sections (Abstract, Introduction, theoretical development, proof sketches, applications, conclusion, and appendix) and across all stages from initial drafting to final revision. The tools were used for boilerplate generation, prose and notation refinement, LaTeX/structure cleanup, translation of informal proof ideas into candidate formal artifacts (Lean/LaTeX), and repeated adversarial reviewer-style critique passes to identify blind spots and clarity gaps.

The author retained full intellectual and editorial control, including problem selection, theorem statements, assumptions, novelty framing, acceptance criteria, and final inclusion/exclusion decisions. No technical claim was accepted solely from AI output. Formal claims reported as machine-verified were admitted only after Lean verification (no `sorry` in cited modules) and direct author review; Lean was used as an integrity gate for responsible AI-assisted research. The author is solely responsible for all statements, citations, and conclusions.

References

[1] C. E. Shannon, "Zero-error capacity of a noisy channel," *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 8–19, 1956.

[2] J. Körner, "Coding of an information source having ambiguous alphabet and the entropy of graphs," *Transactions of the 6th Prague Conference on Information Theory*, pp. 411–425, 1973.

[3] L. Lovász, "On the shannon capacity of a graph," *IEEE Transactions on Information Theory*, vol. 25, no. 1, pp. 1–7, 1979.

[4] H. S. Witsenhausen, "The zero-error side information problem and chromatic numbers," *IEEE Transactions on Information Theory*, vol. 22, no. 5, pp. 592–593, 1976.

[5] D. Slepian and J. K. Wolf, "Noiseless coding of correlated information sources," *IEEE Transactions on Information Theory*, vol. 19, no. 4, pp. 471–480, 1973.

[6] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley-Interscience, 2006.

[7] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*. Springer, 2021, pp. 625–635.

[8] R. M. Fano, *Transmission of Information: A Statistical Theory of Communications*. Cambridge, MA: MIT Press, 1961.

[9] H. S. Witsenhausen and A. D. Wyner, "A conditional entropy bound for a pair of discrete random variables," *IEEE Transactions on Information Theory*, vol. 21, no. 5, pp. 493–501, 1975.

[10] J. Oxley, *Matroid Theory*, 2nd ed. Oxford University Press, 2011.

[11] A. Orlitsky and J. R. Roche, "Coding for computing," *IEEE Transactions on Information Theory*, vol. 47, no. 3, pp. 903–917, 2001.

[12] V. Doshi, D. Shah, M. Médard, and M. Effros, "Functional compression through graph coloring," *IEEE Transactions on Information Theory*, vol. 56, no. 8, pp. 3901–3917, 2010.

[13] K. V. Rashmi, N. B. Shah, K. Ramchandran, and D. Gu, "Multi-version coding—an information-theoretic perspective of consistent distributed storage," *IEEE Transactions on Information Theory*, vol. 63, no. 6, pp. 4111–4128, 2017.

[14] G. Kiczales, J. des Rivières, and D. G. Bobrow, *The Art of the Metaobject Protocol*. MIT Press, 1991.

[15] B. C. Smith, "Reflection and semantics in lisp," in *Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, 1984, pp. 23–35.

[16] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[17] M. Teich, R. Hettinger, and Y. Selivanov, "PEP 487 – simpler customisation of class creation," Python Enhancement Proposals, 2016, online: peps.python.org/pep-0487/.

[18] Python Software Foundation, "Python data model," Python 3 Documentation, 2025, [Online]. Available: docs.python.org/3/reference/datamodel.html.

[19] The PostgreSQL Global Development Group, "Materialized views," PostgreSQL documentation, 2025, [Online]. Available: www.postgresql.org/docs/current/rules-materializedviews.html.

[20] ——, "pg_matviews," PostgreSQL system catalog documentation, 2025, [Online]. Available: www.postgresql.org/docs/current/view-pg-matviews.html.

[21] The Rust Team, "The Rust reference," Language reference, 2024, online: doc.rust-lang.org/reference/.

[22] Microsoft, "Decorators," TypeScript Handbook, 2025, [Online]. Available: www.typescriptlang.org/docs/handbook/decorators.html.

[23] MDN Web Docs, "Classes," JavaScript reference, 2025, [Online]. Available: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes.

[24] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman, *The Java Language Specification, Java SE 17 Edition*. Oracle America, Inc., 2021, online: docs.oracle.com/javase/specs/.

[25] The Go Authors, "The Go programming language specification," Language specification, 2024, online: go.dev/ref/spec.

[26] npm, Inc., "package-lock.json," npm CLI documentation, 2026, [Online]. Available: docs.npmjs.com/cli/v8/configuring-npm/package-lock-json/.

[27] ——, "npm explain," npm CLI documentation, 2026, [Online]. Available: docs.npmjs.com/cli/v7/commands/npm-explain/.

[28] ——, "npm ls," npm CLI documentation, 2026, [Online]. Available: docs.npmjs.com/cli/v7/commands/npm-ls/.

[29] The Rust Project Developers, "Cargo.toml vs cargo.lock," The Cargo Book, 2026, [Online]. Available: doc.rust-lang.org/cargo/guide/cargo-toml-vs-cargo-lock.html.

[30] ——, "cargo-tree," The Cargo Book, 2026, [Online]. Available: doc.rust-lang.org/cargo/commands/cargo-tree.html.

[31] ——, "cargo-metadata," The Cargo Book, 2026, [Online]. Available: doc.rust-lang.org/cargo/commands/cargo-metadata.html.

[32] Poetry Contributors, "CLI," Poetry documentation, 2026, [Online]. Available: python-poetry.org/docs/cli/.

[33] pnpm Contributors, "pnpm install," pnpm documentation, 2026, [Online]. Available: pnpm.io/cli/install.

[34] ——, "pnpm why," pnpm documentation, 2026, [Online]. Available: pnpm.io/cli/why.

[35] ——, "pnpm list," pnpm documentation, 2026, [Online]. Available: pnpm.io/cli/list.

Appendix

This appendix provides a consolidated classification of representative systems according to the architectural patterns identified by Corollary XII.10. The pattern labels used in the table are: Pattern A (Blind Update), meaning missing causal propagation; Pattern B (Amnesiac Derivation), meaning missing provenance observability; and Pattern C (Coherent Kernel), meaning both conditions are present and verifiable rate 1 is realizable.

Host family	Host / Pattern	Class	Prop.	Prov.	Rate 1	Mechanism
Prog. languages	Python [17], [18]	C	yes	yes	yes	For the structural facts considered here, class-definition hooks and runtime hierarchy introspection are host-native.
Databases	Engine-maintained materialized view [19], [20]	C	yes	yes	yes	The engine maintains the derived relation and exposes catalog metadata for it.
Prog. languages	Rust [21]	B	partial	no	no	Compile-time macro generation exists, but the compiled runtime artifact does not retain derivation provenance.
Prog. languages	TypeScript / JavaScript [22], [23]	B	partial	no	no	Decorators can attach metadata, but type erasure and missing subclass provenance block exact verification in the host runtime.
Prog. languages	Java [24]	A/B	no	no	no	Under the runtime-only host interpretation used here, annotations and related derivation tooling are external to the JVM runtime.
Prog. languages	Go [25]	A/B	no	no	no	Reflection exists, but there are no definition-time hooks or host-native implementer registries.
Databases	External ETL copy / duplicate summary table	A/B	no	no	no	Synchronization is externalized, and provenance is no longer intrinsic to the host database.
Deps.	npm [26]–[28]	A	no	yes	no	Lockfile and query commands expose provenance, but manifest edits require explicit npm operations before the derived state is updated.
Deps.	Cargo [29]–[31]	A	no	yes	no	Resolved-dependency provenance is exposed, but `Cargo.lock` remains a distinct derived artifact synchronized through Cargo commands.
Deps.	Poetry [32]	A	no	yes	no	The resolved graph is queryable, but lock regeneration is an explicit step rather than part of the source edit itself.
Deps.	pnpm [33]–[35]	A	no	yes	no	Provenance is queryable, but the lockfile remains separately maintained rather than automatically updated with source edits.

Table A.1. Classification of representative systems. The same theorem-level coordinates apply uniformly across programming-language runtimes, databases, and dependency managers, so the systems analysis is a single validation table rather than a domain-by-domain survey.

Claim Mapping to Lean Handles

Paper handle	Status	Lean support
cor: integrity-threshold	Full	COH2, COH1
cor:matroid-capacity-bounds	Full	FM3, FM6, FM7, FM8, FM9, FM10, FM11
lem:confusability-clique	Full	OM2
lem:info-dof	Full	CC3
thm: affine-fact-matroid	Full	FM1, FM2, FM4, FM5, FM12
thm:asymptotic-capacity	Full	MF7, MF13, MF14, MF15, MF16, MF17, MF22, MF23
thm: causal-necessary	Full	REQ2
thm: coherence-capacity	Full	CC4, CAP3, CAP2
lem:confusability-converse	Full	OM2
lem: derivation-excludes	Full	CC1
lem:dof-gt-one-incoherence	Full	COH2
lem: dof-one-coherence	Full	COH1
lem:equivalence-viewpoint	Full	CC2, OM1, OM2, OM5, OM6, OM7, OM8, OM10, OM11
lem:fano-converse	Full	CC2
lem:finite-error-budgeted	Full	OM3, OM4
lem:independence	Full	IND1, IND2
lem:lower-bound	Full	BND2
lem:multifact-colorability	Full	MF8, MF9, MF12
lem:multifact-max-success	Full	MF2, MF10
lem:multifact-nonclique	Full	MF1, MF3, MF4, MF5, MF6
lem: multifact-product	Full	MF18, MF19, MF20, MF21
lem:multifact-success-set	Full	MF11
lem:pair-injective	Full	OM8, OM9
lem:provenance-necessary	Full	REQ3
lem:side-info	Full	CC5
lem:ssot-iff	Full	L1
lem:timing-forces	Full	REQ1, TRI1
lem: unbounded-gap	Full	BND3
lem:upper-bound	Full	BND1

Auto summary: mapped 29/29 (full=29, derived=0, unmapped=0).

Mechanized Verification (Lean 4)

The main theorem chains are machine-checked in Lean 4, and the cited proof modules compile with zero `sorry` placeholders. The paper keeps some proofs abbreviated for readability, but full formal proofs of the cited results appear in the supplementary Lean artifact.

For the asymptotic arguments, the artifact formalizes the relevant block power-rate sequences directly and proves the required supermultiplicative or subadditive inequalities on those sequences before invoking Fekete-style convergence lemmas. The asymptotic capacity and asymptotic upper values are therefore obtained inside the formal development from explicit sequence objects rather than only quoted as external limits. The cited asymptotic theorems use classical reasoning in the same way as the corresponding finite graph and entropy arguments.

The artifact covers the theorem packages used in the paper:

- the threshold and unified-converse package: coherence, independent rate, the side-information lower bound, pair injectivity, clique/fiber/global budget bounds, conditional-entropy and decoder-output reformulations, deterministic data processing, and the budgeted finite-error extension;
- the graph and asymptotic-capacity package: an explicit confusability graph object, exact recovery as colorability, exact success-set cardinality and weighted success mass via largest colorable induced subgraphs, strong-power identification for block confusability, supermultiplicative block growth, Fekete-style asymptotic Shannon capacity, complement-chromatic upper bounds, and a fixed Lovász-ϑ upper theory with standard orthonormal, primal-PSD, and dual forms;
- the equality package: the clique-fiber equality theorem, export back to the original view-family model through transitivity of confusability, the meet-witness sufficient condition, and the stronger fiber-coherence collapse criterion;
- the affine fact-matroid package: semantic determination as span membership of coordinate functionals, the representable matroid on fact indices, and the identification of minimal determining fact sets with the matroid bases;

- the downstream consequence package: derivation, the timing constraint, causal propagation, provenance observability, the realizability iff theorem, and the $O(1)$ / $\Omega(n)$ rate-complexity gap.

The supplementary artifact contains the theorem-to-declaration coverage matrix, proof inventory, and build instructions.

Lean Handle Index.

AFM1 FactMatroid.determinesFact_iff_mem_factSpan
`Ssot/FactMatroid.lean`
AFM2 FactMatroid.factMatroid_indep_iff
`Ssot/FactMatroid.lean`
AFM3 FactMatroid.basisFacts_card_eq_finrank
`Ssot/FactMatroid.lean`
AFM4 FactMatroid.basisFacts_minimal
`Ssot/FactMatroid.lean`
AFM5 FactMatroid.minimalDetermining_basis
`Ssot/FactMatroid.lean`
AFM6 FactMatroid.factRankFinset_le_card
`Ssot/FactMatroid.lean`
AFM7 FactMatroid.coordProjection_range_le_card
`Ssot/FactMatroid.lean`
AFM8 FactMatroid.factRankFinset_eq_card_of_indepFacts
`Ssot/FactMatroid.lean`
AFM9 FactMatroid.factRankFinset_eq_total_of_determinesAllFinset
`Ssot/FactMatroid.lean`
AFM10 FactMatroid.factSpanFinset_eq_range_coordProjection_dualMap
`Ssot/FactMatroid.lean`
AFM11 FactMatroid.factRankFinset_eq_finrank_range_coordProjection
`Ssot/FactMatroid.lean`
AFM12 FactMatroid.factRankFinset_le_finrank_directionSpace
`Ssot/FactMatroid.lean`
AFM13 FactMatroid.coordProjection_range_eq_card_of_indepFacts
`Ssot/FactMatroid.lean`
AFM14 FactMatroid.coordProjection_range_eq_total_of_determinesAllFinset
`Ssot/FactMatroid.lean`
BND1 ssot_upper_bound
`Ssot/Bounds.lean`
BND2 non_ssot_lower_bound
`Ssot/Bounds.lean`
BND3 ssot_advantage_unbounded
`Ssot/Bounds.lean`
CAP2 ssot_guarantees_coherence
`Ssot/Coherence.lean`
CAP3 non_ssot_permits_incoherence
`Ssot/Coherence.lean`
CC1 CC.derivation_preserves_coherence_core
CC2 CC.finite_counting_converse
CC3 CC.info_dof_counting_contradiction
CC4 CC.rate_incoherence_step
CC5 CC.side_information_requirement
CIA1 ClaimClosure.side_information_requirement
`Ssot/ClaimClosure.lean`
CIA3 ClaimClosure.finite_counting_converse
`Ssot/ClaimClosure.lean`
CIA4 ClaimClosure.info_dof_counting_contradiction
`Ssot/ClaimClosure.lean`
COH1 dof_one_implies_coherent
`Ssot/Coherence.lean`
COH2 dof_gt_one_incoherence_possible
`Ssot/Coherence.lean`
DER2 ClaimClosure.derivation_preserves_coherence_core
`Ssot/ClaimClosure.lean`
FM1 FM.basisFacts_card_eq_finrank
FM2 FM.basisFacts_minimal
FM3 FM.coordProjection_range_le_card
FM4 FM.determinesFact_if_mem_factSpan
FM5 FM.factMatroid_indep_if
FM6 FM.factRankFinset_eq_card_of_indepFacts
FM7 FM.factRankFinset_eq_finrank_range_coordProjection
FM8 FM.factRankFinset_eq_total_of_determinesAllFinset
FM9 FM.factRankFinset_le_card
FM10 FM.factRankFinset_le_finrank_directionSpace
FM11 FM.factSpanFinset_eq_range_coordProjection_dualMap
FM12 FM.minimalDetermining_basis
IND1 both_requirements_independent
`Ssot/Requirements.lean`
IND2 both_requirements_independent'
`Ssot/Requirements.lean`
L1 matroid_basis_equicardinality
`paper1/axis_framework.lean`
L2 ssot_if
MF1 MF.binaryViews_colorable_two
MF2 MF.binaryViews_maxColorableCard_one_eq_two
MF3 MF.binaryViews_nonclique_witness
MF4 MF.binaryViews_not_colorable_one
MF5 MF.binaryViews_oneTag_exactOn_card_le_two
MF6 MF.binaryViews_oneTag_uniform_successProb_le_half
MF7 MF.blockRate_le_shannonCapacityReal
MF8 MF.colorable_if_graphColorable
MF9 MF.exactRecovery_if_properColoring
MF10 MF.exists_exactOn_card_eq_maxColorableCard
MF11 MF.exists_exactOn_if_colorableOn
MF12 MF.exists_exactRecovery_if_colorable
MF13 MF.graphNegLogSeq_subadditive
MF14 MF.graphPowerRate_le_graphShannonCapacityReal
MF15 MF.graphShannonCapacityReal_nonneg
MF16 MF.graphShannonLowerCapacityReal_eq_ofReal_graphShannonCapacityReal
MF17 MF.iSup_graphPowerRate_eq_graphShannonCapacityReal
MF18 MF.pairColorable_of_colorable
MF19 MF.pairProperColoring_of_componentColorings
MF20 MF.pair_product_clique_budget_lower_bound
MF21 MF.pair_product_success_card_lower_bound
MF22 MF.shannonCapacityReal_eq_iSup_blockRate
MF23 MF.tendsto_graphPowerRateNat_graphShannonCapacityReal
MFT3 MultiFact.exactRecovery_iff_properColoring
`Ssot/MultiFact.lean`
MFT4 MultiFact.exists_exactRecovery_iff_colorable
`Ssot/MultiFact.lean`
MFT5 MultiFact.binaryViews_nonclique_witness
`Ssot/MultiFact.lean`
MFT6 MultiFact.binaryViews_colorable_two
`Ssot/MultiFact.lean`
MFT7 MultiFact.binaryViews_not_colorable_one
`Ssot/MultiFact.lean`
MFT8 MultiFact.binaryViews_oneTag_uniform_successProb_le_half
`Ssot/MultiFact.lean`
MFT9 MultiFact.exists_exactOn_iff_colorableOn
`Ssot/MultiFact.lean`
MFT10 MultiFact.binaryViews_oneTag_exactOn_card_le_two
`Ssot/MultiFact.lean`
MFT11 MultiFact.pairProperColoring_of_componentColorings
`Ssot/MultiFact.lean`
MFT12 MultiFact.pairColorable_of_colorable
`Ssot/MultiFact.lean`
MFT13 MultiFact.pair_product_clique_budget_lower_bound
`Ssot/MultiFact.lean`
MFT14 MultiFact.exists_exactOn_card_eq_maxColorableCard
`Ssot/MultiFact.lean`
MFT15 MultiFact.binaryViews_maxColorableCard_one_eq_two
`Ssot/MultiFact.lean`
MFT16 MultiFact.pair_product_success_card_lower_bound
`Ssot/MultiFact.lean`
MFT17 MultiFact.pair_product_independent_lower_bound
`Ssot/MultiFact.lean`
MFT18 MultiFact.exactOn_mass_le_maxColorableMass
`Ssot/MultiFact.lean`
MFT19 MultiFact.exists_exactOn_mass_eq_maxColorableMass
`Ssot/MultiFact.lean`
MFT20 MultiFact.colorable_iff_graphColorable
`Ssot/MultiFact.lean`
MFT21 MultiFact.block_power_independent_lower_bound
`Ssot/MultiFact.lean`
MFT22 MultiFact.block_add_independent_lower_bound
`Ssot/MultiFact.lean`
MFT23 MultiFact.exactRateDistortionValue_upper
`Ssot/MultiFact.lean`
MFT24 MultiFact.exists_exactRateDistortionValue
`Ssot/MultiFact.lean`
MFT25 MultiFact.pairConfusable_iff_strongProdAdj
`Ssot/MultiFact.lean`
MFT26 MultiFact.pairConfusabilityGraph_eq_strongProd
`Ssot/MultiFact.lean`
MFT29 MultiFact.concatBlockConfusable_iff_strongProdAdj
`Ssot/MultiFact.lean`
MFT30 MultiFact.blockConfusabilityGraph_addIsoStrongProd
`Ssot/MultiFact.lean`
MFT31 MultiFact.blockMaxIndependentCard_eq_graphMaxIndependentCard
`Ssot/MultiFact.lean`
MFT32 MultiFact.shannonLowerCapacity_eq_iSup_graphRates
`Ssot/MultiFact.lean`
MFT33 MultiFact.blockRate_le_blockRate_mul
`Ssot/MultiFact.lean`
MFT34 MultiFact.blockConfusable_iff_strongPowAdj
`Ssot/MultiFact.lean`
MFT35 MultiFact.blockConfusabilityGraph_eq_strongPow
`Ssot/MultiFact.lean`
MFT36 MultiFact.shannonLowerCapacity_eq_graphShannonLowerCapacity
`Ssot/MultiFact.lean`
MFT38 MultiFact.strongPow_addIsoStrongProd
`Ssot/MultiFact.lean`
MFT39 MultiFact.graphMaxIndependentCard_strongPow_add_eq_strongProd
`Ssot/MultiFact.lean`
MFT40 MultiFact.graphMaxIndependentCard_strongProd_lower_bound
`Ssot/MultiFact.lean`
MFT41 MultiFact.graphMaxIndependentCard_strongPow_mul_lower_bound
`Ssot/MultiFact.lean`
MFT42 MultiFact.graphPowerRate_le_graphPowerRate_mul
`Ssot/MultiFact.lean`
MFT43 MultiFact.graphNegLogSeq_subadditive
`Ssot/MultiFact.lean`
MFT44 MultiFact.tendsto_graphPowerRateNat_graphShannonCapacityReal
`Ssot/MultiFact.lean`
MFT45 MultiFact.graphPowerRate_le_graphShannonCapacityReal
`Ssot/MultiFact.lean`
MFT46 MultiFact.graphShannonCapacityReal_nonneg
`Ssot/MultiFact.lean`
MFT47 MultiFact.iSup_graphPowerRate_eq_graphShannonCapacityReal
`Ssot/MultiFact.lean`
MFT48 MultiFact.shannonCapacityReal_eq_iSup_blockRate
`Ssot/MultiFact.lean`
MFT49 MultiFact.blockRate_le_shannonCapacityReal
`Ssot/MultiFact.lean`
MFT50 MultiFact.compl_strongPow_adj_iff_exists
`Ssot/MultiFact.lean`
MFT51 MultiFact.complStrongPow_colorable
`Ssot/MultiFact.lean`
MFT52 MultiFact.graphMaxIndependentCard_strongPow_le_complChromaticPow
`Ssot/MultiFact.lean`
MFT53 MultiFact.graphPowerRate_le_log_complChromatic
`Ssot/MultiFact.lean`
MFT54 MultiFact.graphShannonCapacityReal_le_log_complChromatic
`Ssot/MultiFact.lean`
MFT55 MultiFact.graphShannonCapacityReal_le_log_complChromaticNumber
`Ssot/MultiFact.lean`
MFT56 MultiFact.graphShannonLowerCapacity_eq_ofReal_graphShannonCapacityReal
`Ssot/MultiFact.lean`
MFT57 MultiFact.graphThetaPSDLogUpper_eq_graphThetaLogUpper
`Ssot/MultiFact.lean`
MFT58 MultiFact.graphShannonCapacityReal_le_graphThetaPSDLogUpper
`Ssot/MultiFact.lean`
MFT59 MultiFact.graphThetaPSDLogUpper_strongProd_le
`Ssot/MultiFact.lean`
MFT60 MultiFact.graphThetaPSDLogSeq_subadditive
`Ssot/MultiFact.lean`

MFT61 MultiFact.tendsto_graphThetaPSDPowerRateNat_graphThetaPSDAsymptoticUpper
Ssot/MultiFact.lean
MFT62 MultiFact.graphThetaPSDAsymptoticUpper_le_graphThetaPSDLogUpper
Ssot/MultiFact.lean
MFT63 MultiFact.iInf_graphThetaPSDPowerRate_eq_graphThetaPSDAsymptoticUpper
Ssot/MultiFact.lean
MFT64 MultiFact.lovaszOrthoLogUpper_eq_graphThetaLogUpper
Ssot/MultiFact.lean
MFT65 MultiFact.lovaszThetaPrimalLogUpper_eq_graphThetaPSDLogUpper
Ssot/MultiFact.lean
MFT66 MultiFact.graphThetaLiftedDualLogUpper_eq_graphThetaSchurDualLogUpper
Ssot/MultiFact.lean
MFT67 MultiFact.graphOneShotLogMaxIndependentCard_le_graphThetaLogUpper
Ssot/MultiFact.lean
MFT68 MultiFact.graphPowerRate_le_graphThetaPSDPowerRate
Ssot/MultiFact.lean
MFT69 MultiFact.graphShannonCapacityReal_le_graphThetaPSDAsymptoticUpper
Ssot/MultiFact.lean
MFT70 MultiFact.lovaszThetaPrimalAsymptoticUpper_eq_graphThetaPSDAsymptoticUpper
Ssot/MultiFact.lean
MFT71 MultiFact.graphShannonCapacityReal_le_lovaszThetaPrimalAsymptoticUpper
Ssot/MultiFact.lean
MFT72 MultiFact.graphThetaSchurDualLogUpper_strongProd_le
Ssot/MultiFact.lean
MFT73 MultiFact.graphThetaSchurDualLogUpper_iso_eq
Ssot/MultiFact.lean
MFT74 MultiFact.graphThetaSchurDualLogSeq_subadditive
Ssot/MultiFact.lean
MFT75 MultiFact.tendsto_graphThetaSchurDualPowerRateNat_graphThetaSchurDualAsymptoticUpper
Ssot/MultiFact.lean
MFT76 MultiFact.iInf_graphThetaSchurDualPowerRate_eq_graphThetaSchurDualAsymptoticUpper
Ssot/MultiFact.lean
MFT77 MultiFact.graphThetaSchurDualAsymptoticUpper_le_graphThetaPSDAsymptoticUpper
Ssot/MultiFact.lean
MFT78 MultiFact.graphThetaSchurDualAsymptoticUpper_le_lovaszThetaPrimalAsymptoticUpper
Ssot/MultiFact.lean
MFT79 MultiFact.lovaszThetaDualLogUpper_eq_graphThetaSchurDualLogUpper
Ssot/MultiFact.lean
MFT80 MultiFact.lovaszThetaDualAsymptoticUpper_eq_graphThetaSchurDualAsymptoticUpper
Ssot/MultiFact.lean
MFT81 MultiFact.lovaszThetaDualAsymptoticUpper_le_lovaszThetaPrimalAsymptoticUpper
Ssot/MultiFact.lean
MFT82 MultiFact.lovaszThetaDualLogUpper_eq_lovaszThetaPrimalLogUpper
Ssot/MultiFact.lean
MFT83 MultiFact.lovaszThetaDualAsymptoticUpper_eq_lovaszThetaPrimalAsymptoticUpper
Ssot/MultiFact.lean
MFT84 MultiFact.graphShannonCapacityReal_le_lovaszThetaAsymptoticUpper
Ssot/MultiFact.lean
MFT85 MultiFact.graphShannonCapacityReal_eq_lovaszThetaAsymptoticUpper_clusterGraph
Ssot/MultiFact.lean
MFT86 MultiFact.graphShannonCapacityReal_eq_log_card_clusterGraph
Ssot/MultiFact.lean
MFT87 MultiFact.lovaszThetaAsymptoticUpper_eq_log_card_clusterGraph
Ssot/MultiFact.lean
MFT88 MultiFact.confusabilityGraph_eq_clusterGraph_transcriptLabel
Ssot/MultiFact.lean
MFT89 MultiFact.shannonCapacityReal_eq_shannonLovaszThetaAsymptoticUpper_of_fiberCoherent
Ssot/MultiFact.lean
MFT90 MultiFact.shannonCapacityReal_eq_log_transcriptFiberCard_of_fiberCoherent
Ssot/MultiFact.lean
MFT91 MultiFact.shannonLovaszThetaAsymptoticUpper_eq_log_transcriptFiberCard_of_fiberCoherent
Ssot/MultiFact.lean
MFT96 MultiFact.shannonLovaszThetaAsymptoticUpper_eq_log_connectedComponents_of_confusableTransitive
Ssot/MultiFact.lean
MFT100 MultiFact.shannonCapacityReal_eq_shannonLovaszThetaAsymptoticUpper_of_meetWitnessed
Ssot/MultiFact.lean
MFT101 MultiFact.shannonCapacityReal_eq_log_connectedComponents_of_meetWitnessed
Ssot/MultiFact.lean
MFT102 MultiFact.shannonLovaszThetaAsymptoticUpper_eq_log_connectedComponents_of_meetWitnessed
Ssot/MultiFact.lean
MFT107 MultiFact.ViewCompositionClosed
Ssot/MultiFact.lean
MFT108 MultiFact.confusableTransitive_iff_viewCompositionClosed
Ssot/MultiFact.lean
MFT109 MultiFact.confusable_iff_exists_viewClusterAdj
Ssot/MultiFact.lean
MFT110 MultiFact.SupportConfusable
Ssot/MultiFact.lean
MFT111 MultiFact.deterministicSupportConfusable_iff_confusable
Ssot/MultiFact.lean
MFT112 MultiFact.deterministicSupportConfusabilityGraph_eq_confusabilityGraph
Ssot/MultiFact.lean
MFT113 MultiFact.card_state_eq
Ssot/MultiFact.lean
MFT114 MultiFact.confusableOracle_eq_true_iff
Ssot/MultiFact.lean
MFT115 MultiFact.confusableOrderedPairs
Ssot/MultiFact.lean
MFT116 MultiFact.card_state_prod_eq
Ssot/MultiFact.lean
MFT117 MultiFact.card_confusableOrderedPairs_le_naive_bound
Ssot/MultiFact.lean
MFT118 MultiFact.observe_eq_iff_subset_agreeSet
Ssot/MultiFact.lean
MFT119 MultiFact.confusable_iff_exists_view_subset_agreeSet
Ssot/MultiFact.lean
MFT120 MultiFact.agreeSet_statePerm_eq
Ssot/MultiFact.lean
MFT121 MultiFact.confusable_statePerm_iff
Ssot/MultiFact.lean
MFT122 MultiFact.confusable_congr_agreeSet
Ssot/MultiFact.lean
MFT123 MultiFact.confusabilityGraph_statePermIso
Ssot/MultiFact.lean
MFT124 MultiFact.exists_confusabilityGraph_iso_send
Ssot/MultiFact.lean
MFT125 MultiFact.agreementUpwardFamily_upwardClosed
Ssot/MultiFact.lean
MFT126 MultiFact.confusable_iff_agreeSet_mem_agreementUpwardFamily
Ssot/MultiFact.lean
MFT127 MultiFact.confusable_viewFamilyOfAgreementUpwardFamily_iff
Ssot/MultiFact.lean
MFT128 MultiFact.exists_viewFamily_iff_agreementClassified
Ssot/MultiFact.lean
MFT129 MultiFact.agreeSet_eq_univ_iff
Ssot/MultiFact.lean
MFT130 MultiFact.exists_distinct_states_with_agreeSet_eq_iff
Ssot/MultiFact.lean
MFT131 MultiFact.upwardClosed_family_eq_on_proper_subsets_of_same_relation
Ssot/MultiFact.lean
MFT132 MultiFact.agreementOracle_eq_true_iff
Ssot/MultiFact.lean
MFT133 MultiFact.agreementOracle_eq_confusableOracle
Ssot/MultiFact.lean
MFT134 MultiFact.agreementOracleViewWork_le
Ssot/MultiFact.lean
MFT135 MultiFact.agreementOracleTotalWork_le
Ssot/MultiFact.lean
MFT136 MultiFact.meetWitnessed_implies_transitive_confusability
Ssot/MultiFact.lean
MFT137 MultiFact.transitive_confusability_implies_cluster_graph
Ssot/MultiFact.lean
OBS1 ObserverModel.exact_recovery_implies_pair_injective
paper1/Paper1IT/ObserverTagModel.lean
OBS2 ObserverModel.pair_injective_implies_exact_recovery
paper1/Paper1IT/ObserverTagModel.lean
OBS3 ObserverModel.fiber_card_le_tag_alphabet
paper1/Paper1IT/ObserverTagModel.lean
OBS4 ObserverModel.exact_recovery_global_count
paper1/Paper1IT/ObserverTagModel.lean
OBS5 ObserverModel.clique_card_le_tag_alphabet
paper1/Paper1IT/ObserverTagModel.lean
OM1 OM.DeterministicObservable.entropy_coarsen_le_sourceEntropy
OM2 OM.clique_card_le_tag_alphabet
OM3 OM.decodedOutputEntropy_fano_budgeted
OM4 OM.decodedOutputEntropy_fano_observation_only
OM5 OM.decodedOutputEntropy_le_mutualInfoDeterministic
OM6 OM.decodedOutputEntropy_log_gap_nonneg
OM7 OM.deterministicKernel_entropy_data_processing
OM8 OM.exact_recovery_implies_pair_injective
OM9 OM.pair_injective_implies_exact_recovery
OM10 OM.sourceEntropy_eq_observationTagEntropy_add_conditionalEntropyGivenPair
OM11 OM.successSet_clique_entropy_le_log_tags
PRB47 ObserverModel.sourceEntropy_eq_observationTagEntropy_add_conditionalEntropyGivenPair
paper1/Paper1IT/ProbabilisticFinite.lean
PRB55 ObserverModel.successSet_clique_entropy_le_log_tags
paper1/Paper1IT/FanoFinite.lean
PRB68 ObserverModel.deterministicKernel_entropy_data_processing
paper1/Paper1IT/ProbabilisticFinite.lean
PRB73 ObserverModel.observationTagEntropy_gap_nonneg
paper1/Paper1IT/ProbabilisticFinite.lean
PRB74 ObserverModel.observationTagEntropy_gap_eq_zero_of_klDiv_zero_uniform
paper1/Paper1IT/ProbabilisticFinite.lean
PRB81 ObserverModel.decodedOutputEntropy_le_mutualInfoDeterministic
paper1/Paper1IT/ProbabilisticFinite.lean
PRB82 ObserverModel.decodedOutputEntropy_source_gap_nonneg
paper1/Paper1IT/ProbabilisticFinite.lean
PRB83 ObserverModel.decodedOutputEntropy_log_gap_nonneg
paper1/Paper1IT/ProbabilisticFinite.lean
PRB85 ObserverModel.decodedOutputEntropy_fano_budgeted
paper1/Paper1IT/FanoFinite.lean
PRB87 ObserverModel.decodedOutputEntropy_fano_observation_only
paper1/Paper1IT/FanoFinite.lean
PRB90 ObserverModel.DeterministicObservable.entropy_coarsen_le_sourceEntropy
paper1/Paper1IT/ProbabilisticFinite.lean
PRB93 ObserverModel.decodedOutputEntropy_gap_eq_zero_of_klDiv_zero_uniform
paper1/Paper1IT/ProbabilisticFinite.lean
PRB94 ObserverModel.decodedOutputEntropy_gap_pos_implies_klDiv_ne_zero
paper1/Paper1IT/ProbabilisticFinite.lean
PRB95 ObserverModel.decodedOutputObservable_entropy_le_sourceEntropy
paper1/Paper1IT/ProbabilisticFinite.lean
RAT1 ClaimClosure.rate_incoherence_step
Ssot/ClaimClosure.lean
RED1 ClaimClosure.redundancy_incoherence_equiv
Ssot/ClaimClosure.lean
REQ1 structural_facts_fixed_at_definition
Ssot/Foundations.lean
REQ2 definition_hooks_necessary
Ssot/Requirements.lean
REQ3 introspection_necessary_for_verification
Ssot/Requirements.lean
SOT1 ssot_iff
Ssot/Completeness.lean
TRI1 timing_trichotomy_exhaustive
Ssot/Foundations.lean